

*"State is not what your app has -- it is what
your app remembers between one render
and the next."*

MODULE 34

State and Event Management in Angular

Master signals and RxJS to build crm-ui screens that react correctly to data, events, and time.





MODULE 34 OVERVIEW

Making crm-ui's data reactive: signals, RxJS, events, and forms.

crm-ui now has components; this module makes their data live -- reacting to CRM API responses, user input, and time.

DURATION & FOCUS



4 Hours Theory

Signals, RxJS observables, event handling, reactive forms, async state, and change detection performance.



Hands-On Focus

Concept walkthroughs and code patterns you will apply directly when crm-ui labs open later in the course.



Scenario

Make crm-ui's customer list load, filter, and update reactively against the CRM backend's real API.

MODULE ARC



Signals First

Angular's modern reactive primitive for component and shared state -- simple, synchronous, fine-grained.



RxJS for Streams

Async event streams -- HTTP responses, user input over time -- still lean on RxJS's operators.



Forms Need Both

Reactive Forms combine explicit FormGroup/FormControl state with the validation patterns real CRM data needs.

KEY TAKEAWAY Signals handle synchronous component and shared state; RxJS handles asynchronous streams -- most of this module is learning which one fits which job.

WHY STATE MANAGEMENT MATTERS

The exact same customer data can be right on screen A and stale on screen B if state is not managed deliberately.



One Source of Truth

If the customer count lives in three components' local state, updating one and not the others is a bug waiting to happen.



Timing Is Everything

An HTTP response, a click, and a route change can all arrive in an order your code did not expect.



State Bugs Hide Well

A stale loading spinner or a customer list that silently stops updating rarely throws an exception -- it just looks wrong.

KEY TAKEAWAY State management is not optional plumbing -- it is the discipline that keeps what the user sees in sync with what is actually true.

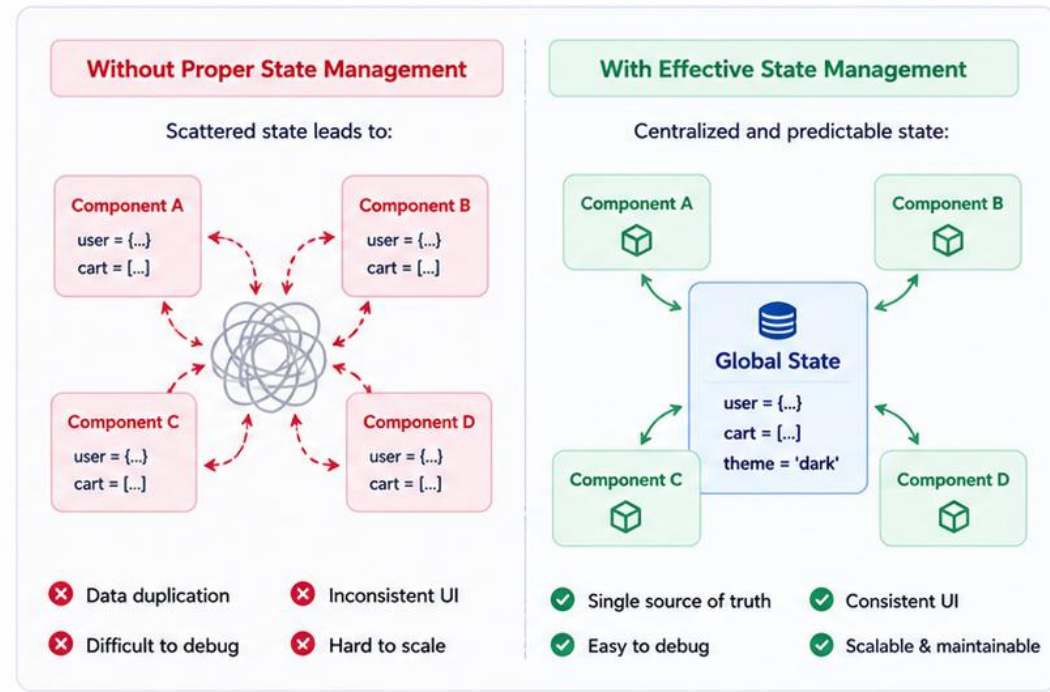
Why State Management Matters

As Angular applications grow, managing data and user interactions becomes complex. Effective state and event management ensures your UI remains **predictable, consistent, and easy to maintain**.



In Modern Angular Applications...

- ✓ Data flows between many components.
- ✓ User actions trigger updates across the UI.
- ✓ Async data and real-time events are common.
- ✓ Without proper state management, complexity increases.



Key Benefits



Predictable UI

State changes follow a clear, predictable flow.



Consistency

All components reflect the same, up-to-date data.



Better Performance

Optimized updates reduce unnecessary re-renders.



Easier Debugging

Centralized state makes issues easier to trace and fix.



Scalability

Supports growing applications and complex features.



Better User Experience

Smoother interactions and real-time responsiveness.



Takeaway: Good state and event management is the backbone of a **robust, maintainable, and user-friendly** Angular application.

MODULE LEARNING OBJECTIVES

By the end of this module you can choose and apply the right reactive tool for any piece of crm-ui state.

Objective	Description
Model State with Signals	Create, read, update, and compute derived values with signal() and computed().
Work with RxJS Streams	Create, subscribe to, and safely unsubscribe from Observables, Subjects, and BehaviorSubjects.
Choose the Right Tool	Justify, for any given piece of crm-ui state, whether a signal or an Observable fits better.
Wire Events and Forms	Handle user events and build Reactive Forms with FormControl/FormGroup validation.
Optimize Change Detection	Apply OnPush and track expressions so crm-ui stays fast as it grows.

KEY TAKEAWAY Every objective here is a skill you will exercise directly in crm-ui -- not abstract theory to memorize and forget.



LEARNING OBJECTIVES

By the end of this module, you will be able to design RESTful APIs that are well-structured, consistent, and aligned with enterprise standards.



Understand REST Principles: Explain what REST is and why it is essential for modern enterprise applications.



Design Resource-Based APIs: Identify resources and design meaningful, consistent, and scalable URI structures.



Use HTTP Effectively: Apply appropriate HTTP methods, status codes, headers, and content types in API communication.



Structure Requests and Responses: Design clear request and response payloads using JSON and proper error handling.



Support Enterprise Requirements: Implement pagination, filtering, sorting, and versioning strategies for real-world APIs.



Document APIs with OpenAPI: Understand and apply OpenAPI (Swagger) for clear and maintainable API documentation.



Follow Best Practices: Apply REST API design best practices and avoid common design mistakes.



UNDERSTANDING UI STATE

Not all state is the same -- naming its category is the first step to managing it correctly.



UI State

Is this dropdown open? Is this row hovered? -
- ephemeral, component-local, never worth persisting.



Domain State

The actual Customer records fetched from the CRM API -- the data the app exists to show.



Session/App State

Is the user logged in? Which theme is active? -- shared across the whole app, outlives any one screen.

KEY TAKEAWAY Naming what kind of state you are looking at -- UI, domain, or session -- tells you immediately where it should live.



UNDERSTANDING UI STATE

UI state is the data that determines what the user interface looks like, how it behaves, and how it responds to user actions.



KEY POINTS

- ✓ **State drives the UI.** When state changes, the UI updates.
- ✓ **State can be local or shared.** Scope determines where it lives.
- ✓ **Angular provides tools** like Signals, RxJS, and Services to manage state effectively.
- ✓ **Good state management** leads to predictable, scalable, and testable apps.

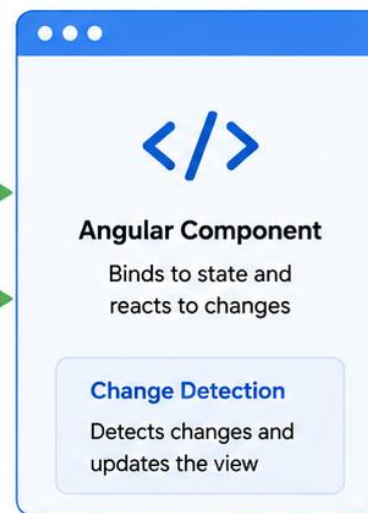
1. STATE SOURCES

Data that affects the UI



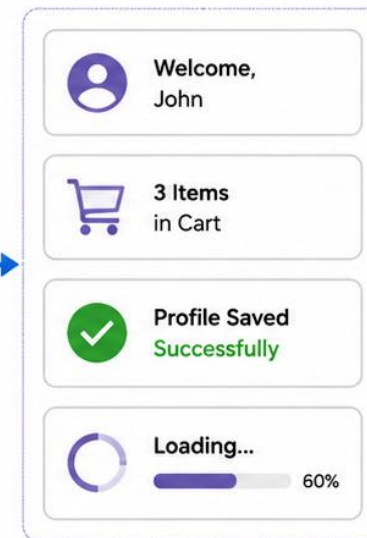
2. UI COMPONENT

Consumes the state



3. RENDERED UI

What the user sees



EXAMPLES OF UI STATE



WHY IT MATTERS

- ✓ Provides a responsive and dynamic user experience
- ✓ Keeps the UI and data in sync
- ✓ Simplifies complex interactions
- ✓ Improves maintainability and scalability



LOCAL VS SHARED STATE

Where state lives should follow directly from how many components actually need to read it.

Aspect	Local State	Shared State
Lives in	A single component (a signal field)	A service (providedIn: 'root' or scoped)
Who can read it	That component and its own children via inputs	Any component that injects the service
crm-ui example	Is this specific card expanded	The current list of Customer records
Risk if placed wrong	Painful prop-drilling through many layers	Unnecessary global coupling, harder to test

KEY TAKEAWAY Default to local state; promote it to a shared service only the moment a second, unrelated component genuinely needs to read it.



LOCAL VS SHARED STATE

→ Scope determines where state lives and who can access it.



LOCAL STATE

Lives inside a component.
Accessible only within that component.



Characteristics

- Scoped to a single component
- Destroyed when component is destroyed
- Simple and easy to manage
- Ideal for UI-specific or temporary data

Example Use Cases



Input text



Toggle switch



Pagination on a page



Modal visibility

```
export class CounterComponent {
  count = 5; // local state
  increment() { this.count++; }
  decrement() { this.count--; }
}
```

Limited Scope

VS

Wide Scope



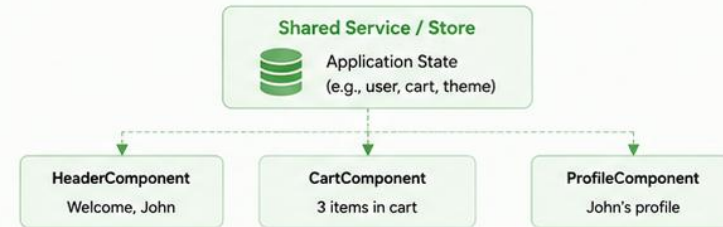
Key Idea

Use local state for component-specific needs and shared state when multiple parts of the app need the same data.



SHARED STATE

Lives in a shared service/store.
Accessible by multiple components.



Characteristics

- Available across multiple components
- Persists as long as the service/store exists
- Centralized and consistent
- Ideal for global or cross-cutting data

Example Use Cases



User information



Shopping cart



Theme settings



Notification state

```
@Injectable({ providedIn: 'root' })
export class AppStateService {
  private user = signal<User | null>(null);
  private cart = signal<CartItem[]>([]);
  getUser() { return this.user(); }
  setUser(u: User) { this.user.set(u); }
}
```



WHEN TO USE WHAT?

Use Local State When:

- ✓ The data is used by only one component
- ✓ The state is UI-specific or temporary
- ✓ You want simple, isolated state management

Use Shared State When:

- ✓ Multiple components need the same data
- ✓ The state should persist across navigation
- ✓ Global or cross-cutting data (auth, theme, cart, etc.)



TIP

Start local. Elevate to shared state only when needed.



ANGULAR REACTIVITY OVERVIEW

Two systems, one goal: make the DOM update automatically whenever the data behind it changes.



Signals

Angular's own fine-grained reactive primitive, stable since v17 -- synchronous, simple, and directly template-aware.



RxJS Observables

A general-purpose library for asynchronous streams, used by Angular's HTTP client, forms, and router internally.



Change Detection

The mechanism that actually walks the component tree and updates the DOM when either system reports a change.

KEY TAKEAWAY Signals and RxJS are complementary, not competing -- and change detection is the engine that turns either kind of change into an updated screen.



ANGULAR REACTIVITY OVERVIEW

Angular provides a reactive programming model that keeps your UI in sync with your data automatically. When state changes, the view updates efficiently.

REACTIVITY PILLARS



Signals (Angular Core)

Fine-grained reactivity for local and shared UI state. Simple, synchronous, and lightweight.



RxJS Observables

Powerful reactive streams for asynchronous events, server data, and complex data flows.



Template Reactivity

Angular automatically tracks dependencies and updates the DOM when data changes.

THE REACTIVE DATA FLOW



1. Change

User action or code updates state

2. State Update

Signal/Observable value changes

3. Change Detection

Angular detects what changed

4. View Update

DOM is updated efficiently

5. UI in Sync

User sees the latest state

WHAT MAKES IT POWERFUL?



Automatic

No manual DOM manipulation



Efficient

Updates only what changed



Predictable

Unidirectional data flow



Developer Friendly

Simple APIs and strong tooling

WHERE REACTIVITY IS USED



Component State

Manage UI state with Signals or Observables



User Events

React to clicks, input, and other interactions



HTTP & WebSocket

Handle async data from APIs and real-time updates



Forms

Keep form state and validation in sync



Navigation & Routing

Respond to route changes and parameters



KEY TAKEAWAYS



Reactivity keeps your UI and data in sync automatically.



Signals are great for local UI state and computed values.



Observables shine for async operations and event streams.



Angular optimizes updates for high performance.



Choose the right tool for each scenario.



INTRODUCTION TO SIGNALS

A signal is a value that knows who is reading it -- and can tell them exactly when it changes.



A Wrapped Value

signal(0) does not return 0 -- it returns a container you call as a function to read the current value.



Automatically Tracked

Any template or computed() that reads a signal is automatically subscribed to its future changes.



Fine-Grained Updates

Only the exact DOM bindings that actually read a changed signal get re-evaluated -- not the whole component.

KEY TAKEAWAY A signal wraps a value and tracks who reads it, so Angular knows exactly what to update the instant that value changes -- no manual subscription bookkeeping required.

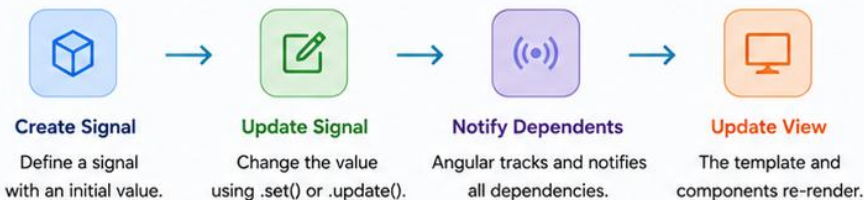
INTRODUCTION TO SIGNALS

Signals are a new reactive primitive in Angular that represent a piece of state. When a signal value changes, Angular automatically updates anything that depends on it.

WHY SIGNALS?

- Simple & Intuitive**
Easy to create and use with a clean API.
- Fine-Grained Reactivity**
Updates only what depends on the signal.
- High Performance**
Minimal overhead and optimized updates.
- Built for Angular**
Deeply integrated with Angular's change detection and template syntax.

HOW SIGNALS WORK



WHERE TO USE SIGNALS?

- Component Local State**
UI state local to a component.
- Shared UI State**
Lightweight state shared across components via services.
- Derived / Computed State**
Values computed from other signals.
- Effects & Side Effects**
Run logic when signals change (e.g., API calls, logging).

BASIC SIGNAL EXAMPLE

TypeScript

```

import { signal } from '@angular/core';

// Create a signal with an initial value
const count = signal(0);

// Read the value
console.log(count()); // 0

// Update the value
count.set(5);
count.update(v => v + 1); // 6

// Use in template
// <p>Count: {{ count() }}</p>
  
```

SIGNALS VS TRADITIONAL APPROACH

Traditional State		With Signals
Manual change detection	→	Automatic & fine-grained
@Input / @Output for sharing	→	Share via services with signals
More boilerplate	→	Less code, cleaner API
Observable + async pipe	→	Simpler synchronous state model



KEY TAKEAWAY

Signals provide a simple, powerful, and performant way to manage reactive state in Angular applications. They reduce complexity and help build more responsive UIs with less code.



CREATING SIGNALS

signal(initialValue) is the one function call that creates a piece of reactive state.



Basic Syntax

`count = signal(0);` creates a `WritableSignal<number>` initialized to 0.



Any Type Works

`signal<Customer[]>([])` or `signal<Customer | null>(null)` -- signals hold objects, arrays, or primitives equally.



Where to Declare

As a class field in a component or service -- typically private, exposed read-only where appropriate.

KEY TAKEAWAY Every signal starts the same way -- a field initialized with `signal(startingValue)` -- whether it holds a number, a `Customer`, or a full array.



CREATING SIGNALS

Signals are created using the `signal()` function from Angular's core package. A signal holds a value and notifies dependents when that value changes.

KEY POINTS

- ✓ A signal holds a value (of any type).
- ✓ You read its value like a function: `signalName()`.
- ✓ You update its value using `.set()` or `.update()`.
- ✓ Signals are reactive and automatically track who depends on them.

SIGNAL SYNTAX

```
const name = signal<type>(initialValue);
```

IMPORT

```
import { signal } from '@angular/core';
```

EXAMPLE IN A COMPONENT

```
counter.component.ts
export class CounterComponent {
  count = signal(0);
  increment() {
    this.count.update(v => v + 1);
  }
}
```



```
counter.component.html
<p>Count: {{ count() }}</p>
<button (click)="increment()">
  Increment
</button>
```

BASIC SIGNAL CREATION EXAMPLE

TypeScript

```
import { signal } from '@angular/core';

// Create signals
const count = signal(0);
const user = signal({
  name: 'John',
  email: 'john@example.com'
});
const isLoggedIn = signal(false);

// Read the values
console.log(count()); // 0
console.log(user().name); // John
console.log(isLoggedIn()); // false

// Update the values
count.set(5); // set a new value
user.update(u => ({ ...u, name: 'Jane' })); // update current
isLoggedIn.set(true); // set to true
```

UPDATING SIGNALS



`.set(value)`
Sets the signal to a new value.

```
count.set(10);
```



`.update(fn)`
Updates the signal based on its current value.

```
count.update(v => v + 1);
```

SUPPORTED TYPES

123

Numbers

```
const age = signal(25);
```

A

Strings

```
const title = signal('Hello');
```

{ }

Objects

```
const user = signal({ name: 'A' });
```

☰

Arrays

```
const items = signal<string[]>([]);
```

✓

Booleans

```
const active = signal(false);
```

INITIAL VALUE IS REQUIRED



Every signal must be initialized with a value.
This value defines the signal's type.

```
const count = signal(0); // ✓
const name = signal<string>(''); // ✓
```



KEY TAKEAWAY

Creating signals is simple. Initialize with a value, read with `()`, and update with `.set()` or `.update()`.



READING SIGNAL VALUES

Call it like a function -- () -- everywhere: in a template, in a computed(), or in plain TypeScript.



In a Template

`{{ customer().name }}` -- the parentheses are required; `{{ customer.name }}` is a common, silent-looking mistake.



In TypeScript

`if (this.isLoading()) { return; }` -- read a signal's current value the same way anywhere in the class.



Reading Registers Interest

Reading inside a template or `computed()` automatically creates a live dependency -- reading elsewhere is just a one-time snapshot.

KEY TAKEAWAY `customer()` always returns the current value at the moment you call it -- the difference between a live dependency and a snapshot is entirely about where that call happens.



READING SIGNAL VALUES

Signal values are accessed by invoking the signal like a function. Reading a signal registers the current context as a dependency, so the UI updates when the value changes.



KEY POINTS

- ✓ Read a signal by calling it: `signalName()`.
- ✓ Reading a signal does not cause side effects.
- ✓ Angular tracks dependencies automatically.
- ✓ When a signal changes, anything that read it will update.

WHERE YOU CAN READ SIGNALS



Templates

Use `{{ signalName() }}` in bindings.



Component Class

Read signals in methods, getters, and logic.



Computed Signals

Use other signals inside computed values.



Effects

Access signals inside effects to react to changes.

BASIC SIGNAL READ EXAMPLE

TypeScript

```
import { signal } from '@angular/core';

const count = signal(0);
const message = signal('Hello');

// Read signal values
console.log(count()); // 0
console.log(message()); // 'Hello'

// Use in expressions
const double = count() * 2;
console.log(double); // 0
```

USING SIGNALS IN COMPONENTS

```
export class ProfileComponent {
  name = signal('Alice');
  age = signal(28);

  greeting() {
    // Reading signals
    return `Hello, ${this.name()}!`;
  }
}
```

In Template (HTML)

```
<h2>{{ name() }}</h2>
<p>Age: {{ age() }}</p>
<p>{{ greeting() }}</p>
```

What You See

**Alice**

Age: 28

Hello, Alice!

EXAMPLE: READING IN DIFFERENT CONTEXTS

```
// 1. In a method
getSummary() {
  return `Count is ${count()}`;
}

// 2. In a computed signal
const isLarge = computed(() => count() > 10);

// 3. In an effect
effect(() => {
  console.log('Count changed to:', count());
});
```

WHAT HAPPENS WHEN YOU READ



You read
a signal



Angular registers
the dependency



Signal value
changes



Angular updates
the dependent view



BEST PRACTICES

- Read signals freely – it's lightweight and safe.
- Use `computed()` for derived values instead of reading the same signal multiple times.
- Avoid reading signals inside heavy loops or unnecessary repeated calls in methods.



UPDATING SIGNALS

Two methods cover every update: `.set()` replaces the value outright; `.update()` derives the new value from the old.

Method	Use It When
<code>.set(newValue)</code>	You already have the complete new value -- e.g. <code>isLoading.set(true)</code> .
<code>.update(current => newValue)</code>	The new value depends on the current one -- e.g. <code>count.update(c => c + 1)</code> .
Arrays/objects: always replace	<code>customers.update(list => [...list, newCustomer])</code> -- never <code>push()</code> into the old array in place.

KEY TAKEAWAY Signals detect a new value by reference for objects and arrays -- always produce a new array/object in `.update()`, never mutate the existing one in place.

UPDATING SIGNALS

Signals are updated using `.set()` to replace the value or `.update()` to compute a new value based on the current one. Updates are reactive and automatically notify all dependents.

KEY POINTS

- ✓ `.set(value)` replaces the signal's value.
- ✓ `.update(fn)` updates the value based on the current value using a function.
- ✓ Updating a signal triggers change detection and notifies dependents.
- ✓ Use `.update()` for safe updates based on the previous value.

WHEN TO USE WHAT?

- `.set()` When you already have the new value.
Example: `count.set(10);`
- `.update()` When the new value depends on the current value.
Example: `count.update(v => v + 1);`

BASIC UPDATE EXAMPLES

```
import { signal } from '@angular/core';

const count = signal(0);
const user = signal({ name: 'Alice', age: 28 });

// 1. Using .set() - replace with a new value
count.set(5); // count is now 5
user.set({ name: 'Bob', age: 30 }); // new object

// 2. Using .update() - compute from current value
count.update(v => v + 1); // 5 -> 6
count.update(v => v + 10); // 6 -> 16
user.update(u => ({ ...u, age: u.age + 1 })); // 30 -> 31

// 3. Using .update() with more logic
count.update(v => (v % 2 === 0 ? v + 1 : v)); // make it odd
user.update(u => ({ ...u, name: u.name.toUpperCase() }));
// 108
```

TypeScript

UPDATE METHODS

`.set(value)`

Replaces the entire value.

```
count.set(10);
user.set({ name: 'John', age: 29 });
```

`.update(fn)`

Updates based on the current value.

```
count.update(v => v + 1);
user.update(u => ({ ...u, age: u.age + 1 }));
```



NOTE
For objects and arrays, always return a new reference in `.update()` to ensure change detection works correctly.

COMMON UPDATE SCENARIOS

Scenario	Recommended	Example
Increment a counter	<code>.update()</code>	<code>count.update(v => v + 1);</code>
Set a specific value	<code>.set()</code>	<code>count.set(100);</code>
Update object property	<code>.update()</code>	<code>user.update(u => ({ ...u, name: 'Alice' }));</code>
Toggle boolean	<code>.update()</code>	<code>active.update(v => !v);</code>
Add item to array	<code>.update()</code>	<code>items.update(arr => [...arr, newItem]);</code>
Replace array	<code>.set()</code>	<code>items.set([1, 2, 3]);</code>

IN TEMPLATE (HTML)

Component
(Updates)

```
count.update(v => v + 1);
count.set(0);
```

UI updates
automatically

Template
(Reads)

```
<p>Count: {{ count() }}</p>
<button (click)="increment()">+1</button>
```

WHAT HAPPENS WHEN YOU UPDATE A SIGNAL?



BEST PRACTICES

- ✓ Prefer `.update()` when the new value depends on the previous value.
- ✓ Avoid mutating objects/arrays directly; return new references.
- ✓ Keep signals small and focused – one piece of state per signal.
- ✓ Let Angular reactivity handle the updates automatically.

✗ Avoid (mutating directly)

```
user.update(u => {
  u.age = u.age + 1;
  return u; // ✗ same reference
});
```

✓ Do (return new object)

```
user.update(u => ({ ...u, age: u.age + 1 }));
```

COMPUTED SIGNALS

A read-only signal derived from other signals -- recalculated automatically, and only when it actually needs to be.



computed(() => ...)

```
activeCustomers = computed(() =>  
  customers().filter(c => c.status === 'active'));
```



Read-Only

A computed signal has no `.set()` or `.update()` -
- it can only change when its own
dependencies change.



Memoized & Lazy

Its function only re-runs when a dependency
actually changes, and only if something is still
reading it.

KEY TAKEAWAY `computed()` lets you derive filtered, sorted, or formatted values from your real state without duplicating or manually resyncing anything.



Computed Signals

Computed signals derive their value from one or more source signals. They automatically update when any of their dependencies change.



Key Points

- Computed signals are read-only.
- They track dependencies automatically.
- They re-compute only when needed.
- Great for deriving values, filtering, aggregations, or UI-ready data.



Example

```
import { signal, computed } from '@angular/core';

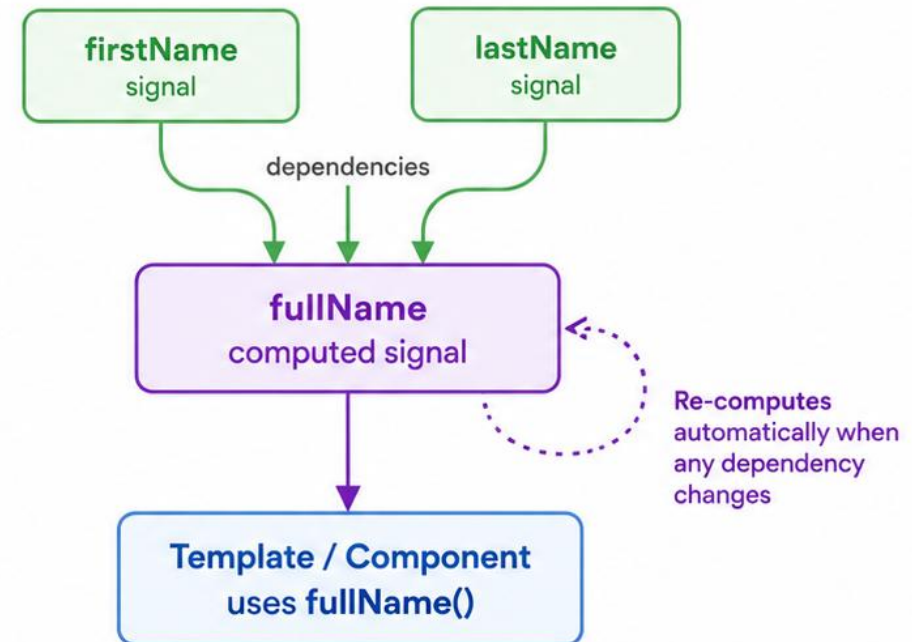
const firstName = signal('John');
const lastName = signal('Doe');

const fullName = computed(() =>
  `${firstName()} ${lastName()}`
);

// fullName() returns "John Doe"
```



How It Works



Why Use Computed Signals?



Automatic
dependency tracking



Efficient
re-computation



Read-only
and predictable



Cleaner templates
and component logic

EFFECTS

effect() runs a side effect automatically whenever any signal it reads changes -- for work outside the template.



effect(() => { ... })

Runs once immediately, then again every time a signal read inside it changes.



For Side Effects Only

Logging, syncing to localStorage, or calling a non-Angular API -- never for computing a value (use computed() instead).



Injection Context Required

effect() must be created in a constructor, field initializer, or via runInInjectionContext().

KEY TAKEAWAY Reach for effect() only when you need to run code the template itself cannot express -- for deriving a value, computed() is almost always the right tool instead.



Effects

Effects run side effects in response to changes in signals. They help you react to state changes and perform operations.



Key Points

- Effects track the signals they read automatically.
- They re-run whenever any of those signals change.
- Used for side effects (logging, calling APIs, updating local storage, etc.).
- **Effects** should not update signals to avoid loops.
- Cleanup function can be returned for subscriptions or timers.



Example

```
import { signal, effect } from '@angular/core';

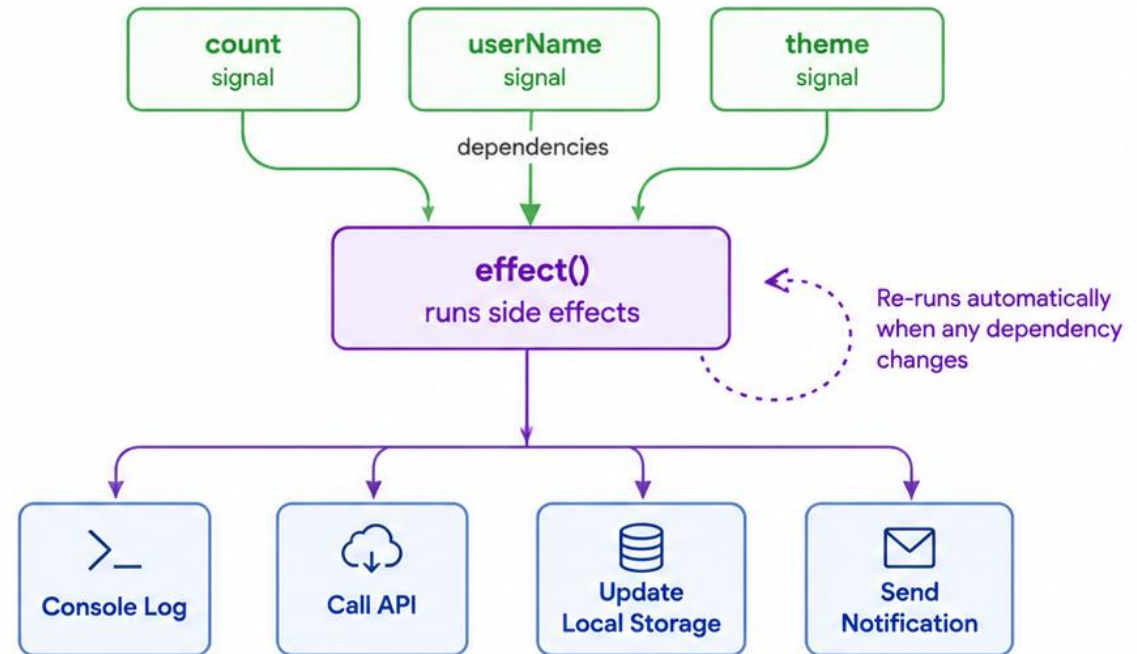
const count = signal(0);
const message = signal('');

effect(() => {
  // This effect depends on 'count'
  console.log(`Count changed: ${count()}`);
  message.set(`Count is ${count()}`);
});

// Updates to 'count' will re-run this effect
count.set(1);
```



How It Works



Common Use Cases



Logging state changes for debugging



Calling APIs when filters or inputs change



Syncing data with local storage/session



Triggering notifications or analytics events

SIGNAL DATA FLOW

Source signals, computed signals, and effects form a directed graph -- data flows one way, always downhill.



Source Signals

Created with `signal()` -- the roots of the graph, updated directly via `.set()/.update()`.



Computed Signals

Derived nodes -- read one or more source (or other computed) signals, recalculate automatically.



Effects

Leaf nodes -- consume signals to run side effects, never feed a value back into the graph.

KEY TAKEAWAY Draw crm-ui's own signal graph on a whiteboard -- source signals at the root, computed signals branching off them, effects as the leaves that never write back.



Signal Data Flow

Signals form a reactive graph. When a source signal changes, updates flow automatically to all dependents.



How It Flows



Key Points

- Signals form a dependency graph.
- Updates flow from sources → computed → effects → UI.
- Only dependents of the changed signal re-run.
- Angular batches updates for performance.



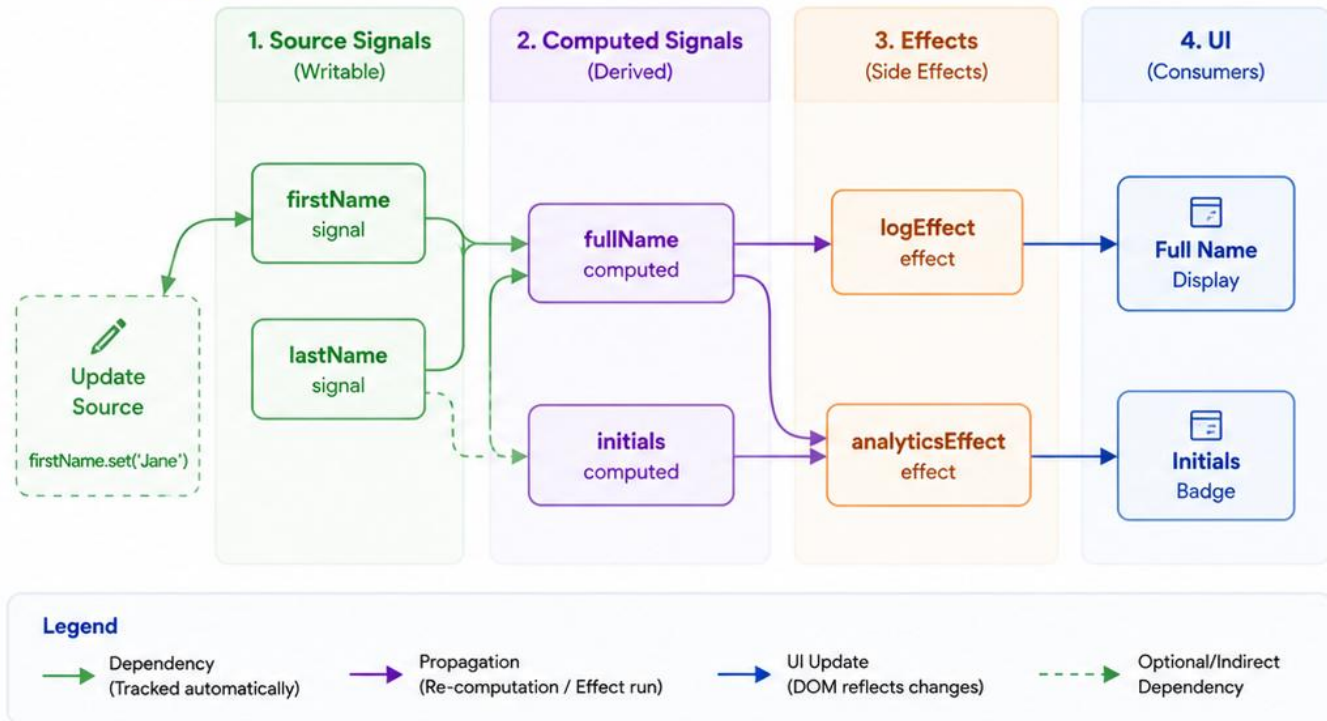
Example

```
const firstName = signal('John');
const lastName = signal('Doe');

const fullName = computed(() =>
  `${firstName()} ${lastName()}`
);

effect(() => {
  console.log('Full name changed:', fullName());
});

// Change flow:
// firstName.set('Jane') → fullName recomputes → effect runs
```



Flow Summary



1. Update Source
A writable signal value changes.



2. Re-compute
Computed signals that depend on it re-compute.



3. Run Effects
Effects that depend on computed signals run.



4. Update UI
The UI automatically reflects the changes.

SIGNALS VS TRADITIONAL COMPONENT STATE

Before signals, component state was just plain class fields -- and the template had no idea when they changed.

Aspect	Plain Class Field	Signal
Declaring it	count = 0;	count = signal(0);
Reading it in a template	{{ count }}	{{ count() }}
Updating it	this.count = this.count + 1;	this.count.update(c => c + 1);
How Angular knows to re-render	Zone.js patches async APIs and triggers a broad check	The signal itself reports exactly what depends on it
Update precision	Angular re-checks a wider portion of the tree	Only bindings that actually read this signal

KEY TAKEAWAY A plain field relies on Angular's Zone.js-based change detection noticing something happened somewhere; a signal tells Angular exactly what changed and what depends on it.

TRY IT YOURSELF — EXERCISE 1: MODEL STATE WITH SIGNALS

Reinforces: local versus shared state, signal creation, reading, updating, and computed values as you just covered them.

STEPS (notes/lab34-signal-state.md)

Step	What To Do
1 — State List	List the signals the customer screen needs: customers, filter text, loading.
2 — Read and Write	Show how a signal is read in a template and updated in a handler.
3 — Computed	Define the filtered list as a computed signal over the other two.
4 — Why Not a Field	Say what a signal gives you that a plain class field does not.

Signal state sketch

```
customers = signal<Customer[]>([])      filter = signal('')
visible   = computed(() => customers().filter(c => match(c, filter())))
update: customers.set(list) / filter.set(text)  read in template: visible()
```

TRY IT NOW — Complete Exercise 1 before continuing: [exercises/exercise-01-signal-state-model.md](#) (workspace: [examples/module-34-exercises](#)).



Signals vs Traditional Component State

Both approaches manage state in Angular, but signals provide a more reactive, fine-grained, and efficient model.



Key Takeaways

- ✓ Signals offer automatic reactivity and fine-grained updates.
- ✓ Traditional state requires manual updates and change detection.
- ✓ Signals reduce boilerplate and improve performance.
- ✓ Use signals for most UI state in modern Angular apps.

	Signals (Angular Signals)	VS	Traditional Component State
Reactivity	✓ Automatic. Updates propagate to dependents.		✗ Manual. Requires Angular change detection.
Updates	✓ Fine-grained. Only affected parts update.		✗ Coarse-grained. Whole component may re-render.
Code Style	✓ Declarative and concise.		✗ Imperative and more boilerplate.
Performance	✓ Better performance with fewer re-renders.		✗ More re-renders and more work.
Best Used For	✓ UI state, derived state, computed values, and reactive flows.		✗ Complex objects, manual control, external library integration.
Mutation Control	✓ Encourages immutable patterns.		✗ Easy to mutate, can cause side effects.

Signals Example

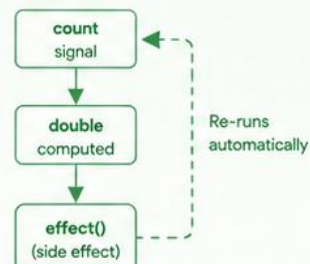
```
import { signal, computed, effect } from '@angular/core';

const count = signal(0);
const double = computed(() => count() * 2);

effect(() => {
  console.log('Count: ${count()}, Double: ${double()}');
});

count.set(1); // Automatically updates computed and effect
```

Data Flow



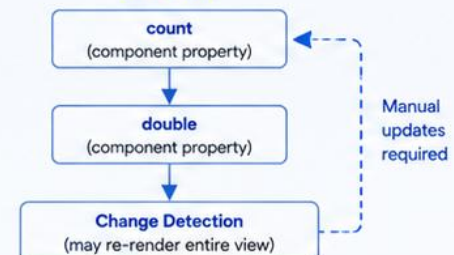
Traditional State Example

```
count = 0;
double = 0;

increment() {
  this.count = this.count + 1;
  this.double = this.count * 2;
}

ngOnChanges() or template binding
triggers change detection.
```

Data Flow



When to Use Which?



Use Signals When:

- UI state is local and reactive
- You need derived or computed values
- You want better performance



Use Traditional State When:

- Managing complex objects
- Interfacing with external libraries
- Needing full control over updates



Best Practice:

Use signals by default. Mix with traditional state only when needed.

INTRODUCTION TO RXJS

A library for working with asynchronous, event-based data as composable streams -- deeply embedded in Angular.



Streams Over Time

An RxJS Observable represents a sequence of values that arrive over time, not just one.



Built Into Angular

HttpClient, the Router, and Reactive Forms all expose their data as Observables.



Operators Compose

map, filter, debounceTime, and switchMap transform a stream declaratively, piped together.

KEY TAKEAWAY RxJS models 'this will produce values over time' as a first-class, composable concept -- essential for HTTP calls, user input streams, and anything genuinely asynchronous.



Introduction to RxJS

RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using **Observables**, enabling you to work with asynchronous data streams.



What is RxJS?

RxJS is a library for composing asynchronous and event-based programs using **Observables** sequences. It provides operators to transform, filter, combine, and control data streams.



Why Use RxJS in Angular?

- Angular's **HttpClient** and many APIs return **Observables**.
- Handles multiple values over time, not just a single value.
- Powerful operators to manipulate and combine streams.
- Better for complex async scenarios like UI events, WebSockets, timers, and more.



Quick Example

```
import { of } from 'rxjs';

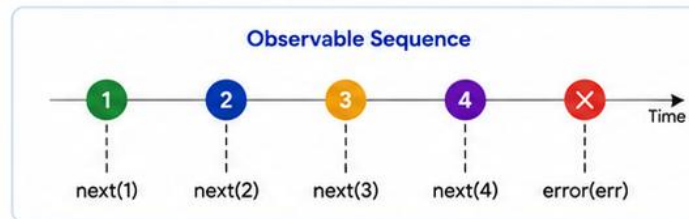
of(1, 2, 3).subscribe(value => {
  console.log(value);
});

// Output: 1 2 3 (emitted over time)
```



Core Concept: Observable

An **Observable** represents a stream of data over time. It can emit multiple values, complete successfully, or error.



Observable Outcomes

- 1 **next(value)** Emits a value to subscribers
- complete()** Stream completes successfully
- error(err)** Stream terminates with an error



Common Creation Functions

Function	Description	Example
of()	Emits a set of values and completes.	<code>of(1, 2, 3)</code>
from()	Converts array, promise, or iterable into an Observable.	<code>from([1, 2, 3])</code>
interval()	Emits incremental numbers at intervals.	<code>interval(1000)</code>
timer()	Emits after a due time and optionally continues at intervals.	<code>timer(1000, 2000)</code>



Where RxJS is Used in Angular



HttpClient
All HTTP requests return Observables.



User Events
Click, input, keyup, mouse events.



Real-time Data
WebSockets, SSE, live updates.



Async Operations
Timers, delays, animations, and more.



Key Takeaway

RxJS empowers Angular to handle asynchronous data streams in a clean, powerful, and reactive way. It forms the backbone of reactive programming in Angular applications.

OBSERVABLE FUNDAMENTALS

An Observable is lazy -- it does nothing at all until something subscribes to it.



Lazy by Default

Calling `http.get(...)` does not send a request -- subscribing to the returned Observable does.



Zero, One, or Many Values

An Observable can emit nothing, one value and complete (typical for HTTP), or many values over time.



Cold vs Hot

A 'cold' Observable (like most HTTP calls) starts its work fresh for every new subscriber.

KEY TAKEAWAY Nothing happens with an Observable until `.subscribe()` is called -- that laziness is a deliberate, powerful design choice, not an inconvenience.



Observable Fundamentals

An **Observable** is a stream of data over time. It can emit multiple values, complete successfully, or terminate with an error.



What is an Observable?

- Represents a lazy, push-based data stream.
- Can emit zero or more values asynchronously.
- Three possible notification types: `next`, `complete`, `error`.
- Nothing happens until someone subscribes.



Basic Example

```
import { Observable } from 'rxjs';

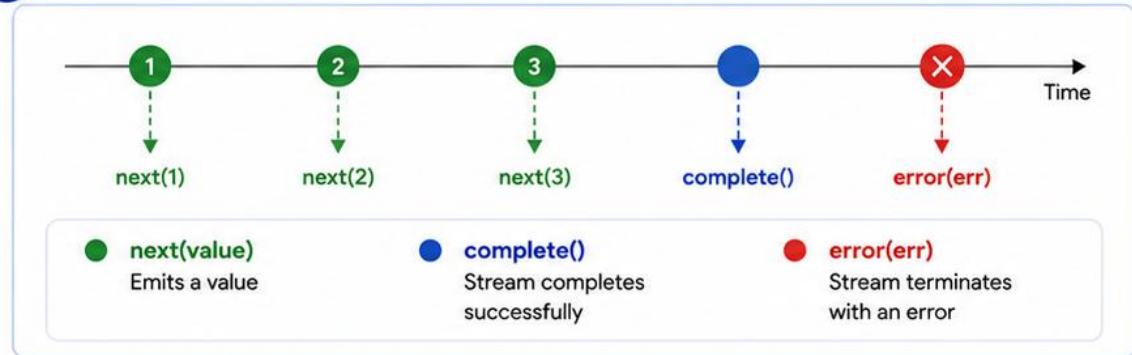
const numbers$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});

// Nothing happens until we subscribe
numbers$.subscribe({
  next: v => console.log('next:', v),
  complete: () => console.log('completed'),
  error: e => console.error('error:', e)
});

// Output:
// next: 1 next: 2 next: 3 completed
```



Observable Lifecycle



Key Characteristics

	Lazy	Execution starts only on <code>subscribe()</code> .
	Push-based	Values are pushed to the consumer.
	Can emit multiple values	Zero, one, or many <code>next()</code> values.
	Terminal events	Either <code>complete()</code> or <code>error()</code> ends the stream.
	Composable	Use operators to transform and combine streams.



Key Takeaway



Observables provide a robust foundation for handling asynchronous data in Angular.

They are the backbone of **HttpClient**, router events, forms, and many other APIs.



Why It Matters



Handles asynchronous data cleanly.



Scales with complex event-driven workflows.



Powerful operators improve productivity.



Consistent model across Angular ecosystem.

OBSERVER AND SUBSCRIPTION MODEL

Subscribing hands an Observable three callbacks: what to do on each value, on error, and on completion.



next(value)

Called once per emitted value -- for an HTTP call, typically exactly once with the response body.



error(err)

Called if the stream fails -- after error fires, no further next() or complete() calls happen.



complete()

Called when the stream is finished emitting -- most HTTP Observables complete immediately after their one value.

KEY TAKEAWAY next, error, and complete are the entire contract between an Observable and whatever is subscribed to it -- learn this shape once and it explains every .subscribe() call you will ever write.



Observer and Subscription Model

In RxJS, data flows from an **Observable** to its **Observers** through a **Subscription**.

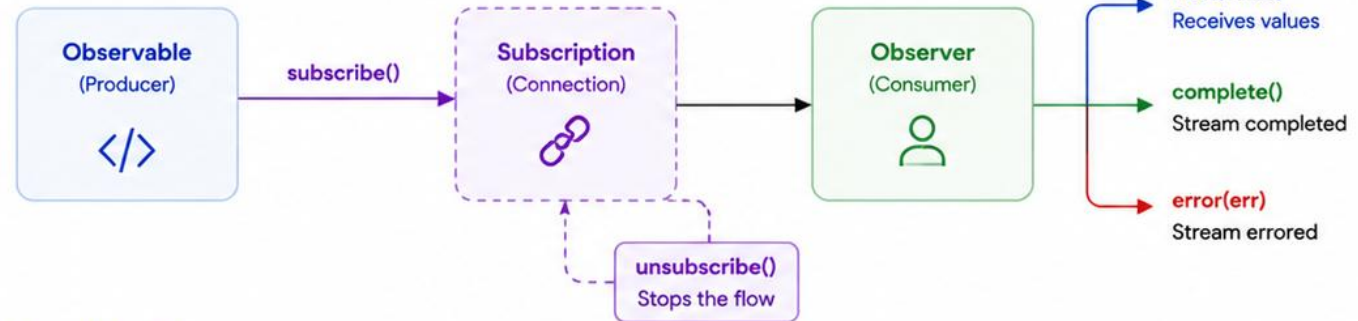


The Players

- Observable (Producer)**
Creates and emits values over time.
- Observer (Consumer)**
Listens for values, completion, or errors.
- Subscription (Connection)**
Represents the active execution.
Unsubscribe to stop it.



How It Works



Code Example

```

import { Observable } from 'rxjs';

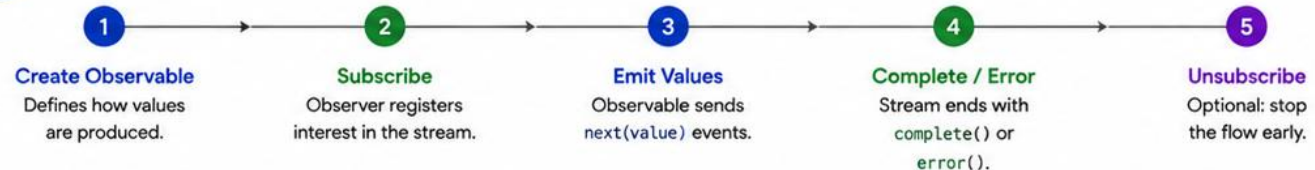
const numbers$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});

const subscription = numbers$.subscribe({
  next: value => console.log('next:', value),
  error: err => console.error('error:'),
  complete: () => console.log('complete')
});

// Later, when you no longer need the stream
subscription.unsubscribe();
  
```



Execution Flow



Key Takeaways



Observer defines what to do on next, complete, and error.



Subscription is the link between Observable and Observer.



Unsubscribe to prevent memory leaks and stop unnecessary work.



Every `subscribe()` creates a new execution of the Observable.



Why It Matters

Understanding the Observer-Subscription model is essential to work with RxJS effectively. It enables reactive, asynchronous, and event-driven programming in Angular applications.

CREATING OBSERVABLES

Angular code rarely constructs an Observable by hand -- it almost always comes from a creation function.

Creation Function	Produces
of(1, 2, 3)	Emits the given values synchronously, in order, then completes.
from(promiseOrArray)	Converts a Promise or array/iterable into an Observable.
interval(1000)	Emits an incrementing number every 1000ms, forever, until unsubscribed.
fromEvent(el, 'click')	Wraps a DOM event as an Observable stream of Event objects.
new Observable(subscriber => {...})	The low-level constructor -- rarely needed directly in Angular application code.

KEY TAKEAWAY HttpClient hands you Observables already built; these creation functions matter most for turning other things -- events, timers, promises -- into Observables you can pipe and combine.



Creating Observables

Observables can be created using a variety of creation functions based on your data source and use case.



Why Create Observables?

- ✓ Represent asynchronous or event-based data.
- ✓ Emit multiple values over time.
- ✓ Integrate with promises, events, timers, and more.
- ✓ Form the foundation for reactive programming in Angular.

Code Examples

```
import { of, from, interval, timer, fromEvent, Observable } from 'rxjs';

// of() - static values
of(1, 2, 3).subscribe(console.log);

// from() - array
from(['A', 'B', 'C']).subscribe(console.log);

// interval() - emits every 1000ms
interval(1000).subscribe(value => console.log('Interval:', value));

// timer() - emits 0 after 2 seconds, then every 1 second
timer(2000, 1000).subscribe(value => console.log('Timer:', value));

// fromEvent() - DOM click events
fromEvent(document, 'click').subscribe(event => console.log('Clicked!', event));

// new Observable() - custom observable
const custom$ = new Observable<number>(subscriber => {
  subscriber.next(10);
  subscriber.next(20);
  subscriber.complete();
  return () => console.log('Clean up');
});
custom$.subscribe(console.log);
```

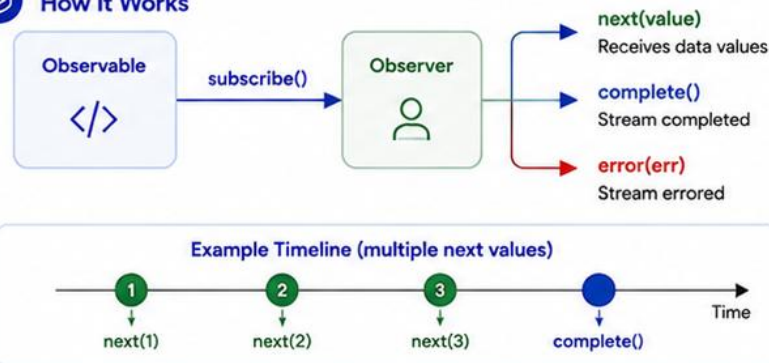


Common Observable Creation Methods

Creation Method	Description	Use Case	Example Output
<code>of()</code>	Emits the provided values and completes.	Static values	1 → 2 → 3 ●
<code>from()</code>	Converts an array, promise, iterable, or async source into an Observable.	Arrays, promises, iterables	1 → 2 → 3 ●
<code>interval()</code>	Emits incremental numbers at specified intervals.	Timers, polling	0 → 1 → 2 → 3 → ...
<code>timer()</code>	Emits a single value or sequence after a due time.	Delayed actions	⌚ --- 0 → 1 → 2 → ...
<code>fromEvent()</code>	Creates an Observable from DOM events.	User interactions	⬮ click → click → ...
<code>new Observable()</code>	Create a custom Observable with full control.	Custom logic, advanced use cases	⚡ next ✓ complete ✗ error



How It Works



Best Practices

- ✓ Choose the right creation method for your use case.
- ✓ Prefer higher-level creation functions (`of`, `from`, `interval`, etc.).
- ✓ Use `new Observable()` only when you need custom logic.
- ✓ Remember to unsubscribe to prevent memory leaks.



Key Takeaway

RxJS provides multiple ways to create Observables depending on your data source. Once created, they can emit multiple values over time and power reactive streams in your Angular applications.

SUBSCRIBING TO OBSERVABLES

Two ways to subscribe in Angular: manually in TypeScript, or declaratively with the async pipe in a template.



Manual .subscribe()

Called in a component method (often inside `ngOnInit`) -- gives full control, but you own cleanup.



The async Pipe

`{{ customers$ | async }}` in a template -- Angular subscribes and unsubscribes for you automatically.



Prefer async Where You Can

Less code, no manual cleanup, and it integrates cleanly with Angular's change detection.

KEY TAKEAWAY The async pipe eliminates an entire category of subscription-management bugs by tying the subscription's lifetime directly to the template that uses it.



Subscribing to Observables



Observables are cold until you subscribe. A subscription activates the Observable and starts receiving the emitted values.



Key Points

- Subscribe to start the Observable execution.
- Handle next, error, and complete events.
- Always unsubscribe to prevent memory leaks.
- Use the async pipe in templates when possible.



Tip: Unsubscribe in `ngOnDestroy` or use the `async` pipe to manage subscriptions automatically.

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { UserService } from '../user.service';

export class UserListComponent implements OnInit, OnDestroy {
  users: User[] = [];
  private subscription: Subscription;

  constructor(private userService: UserService) {}

  ngOnInit(): void {
    this.subscription = this.userService.getUsers().subscribe({
      next: (data) => this.users = data,
      error: (err) => console.error('Error loading users', err),
      complete: () => console.log('Users loaded successfully')
    });
  }

  ngOnDestroy(): void {
    this.subscription.unsubscribe(); // Prevent memory leaks
  }
}
```



Observable Subscription Flow

Edit

Observable
(e.g., HTTP request)

subscribe()



Subscription
(Execution starts)

next() – Receive data

error() – Handle errors

complete() – Execution finished



UNSUBSCRIBING SAFELY

Every manual subscription that outlives a single emission needs an explicit plan for how it gets cleaned up.



takeUntilDestroyed()

An RxJS-interop operator that automatically completes a subscription when the component is destroyed.



The async Pipe

Handles unsubscription automatically -- no manual cleanup code needed at all.



Manual unsubscribe()

Store the Subscription and call `.unsubscribe()` in `ngOnDestroy` -- the old pattern, still valid but easy to forget.

KEY TAKEAWAY `takeUntilDestroyed()` and the async pipe both solve the manual-cleanup problem Module 33 flagged under Component Destruction -- prefer them over remembering `ngOnDestroy` by hand.



Unsubscribing Safely

Prevent memory leaks and unexpected behavior



Why it matters

Observables can continue emitting even after a component is **destroyed**. Always **unsubscribe** to free resources and avoid memory leaks.



Safe Unsubscription Patterns

1

Manual unsubscribe

Store the Subscription and call unsubscribe() in ngOnDestroy.

2

takeUntil + Subject

Use takeUntil with a destroy\$ Subject to auto-unsubscribe.

3

Async pipe

Use the async pipe in templates – Angular unsubscribes automatically.

4

Decorators / Operators

Use operators like takeUntilDestroyed() (Angular v16+) for simplicity.

```
takeUntil + Subject (Recommended)

import { Component, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';
import { UserService } from '../user.service';

@Component({ /* ... */ })
export class UserListComponent implements OnDestroy {
  users: User[] = [];
  private destroy$ = new Subject<void>();

  constructor(private userService: UserService) {}

  ngOnInit(): void {
    this.userService.getUsers()
      .pipe(takeUntil(this.destroy$))
      .subscribe(users => this.users = users);
  }

  ngOnDestroy(): void {
    this.destroy$.next(); // complete the notifier
    this.destroy$.complete(); // release resources
  }
}
```



Best Practices



Unsubscribe in ngOnDestroy for all manual subscriptions.



Prefer async pipe in templates when possible.



Use takeUntil pattern for multiple subscriptions.



Avoid nested subscriptions.



Make it a habit to clean up resources.



Component Lifecycle & Subscription



Component Created
ngOnInit() called



Subscribe
Observable starts emitting values



Component Active
Values are received while component exists



Component Destroyed
ngOnDestroy() called



Unsubscribe
Observable completes and resources are freed



Key Takeaway: Always unsubscribe from Observables you create manually. Use the async pipe or takeUntil pattern to keep your Angular applications memory-efficient and predictable.



Memory leaks lead to performance issues and unexpected behavior.

SUBJECT OVERVIEW

A Subject is both an Observable and an Observer at once -- something else can push values into it manually.



Multicast by Nature

Unlike a cold Observable, every subscriber to a Subject shares the exact same underlying execution.



You Call `.next()` Yourself

`searchTerms.next('acme')` pushes a value into the stream from anywhere in your code.



No Replay for Late Subscribers

A plain Subject only delivers values emitted after a subscriber joins -- earlier values are gone.

KEY TAKEAWAY A Subject is the bridge between imperative code (a click handler, a service method) and the declarative Observable pipelines built around it.



Subject Overview

Subjects are both Observables and Observers.
They multicast values to multiple subscribers.



Multicast
Share values with multiple subscribers



Hot Observable
Starts emitting immediately



Both Roles
Acts as Observable and Observer



State Handling
Different types handle state differently



? What is a Subject?

A Subject is a special type of Observable that allows values to be multicast to many Observers. It also allows values to be pushed into the stream using `next()`.

☰ Key Points

- Subjects are “hot” – they start emitting as soon as values are pushed.
- New subscribers do not receive previous values (except BehaviorSubject and ReplaySubject).
- Use Subjects for event emitters, shared state, and caching.
- Choose the right Subject type based on your use case.

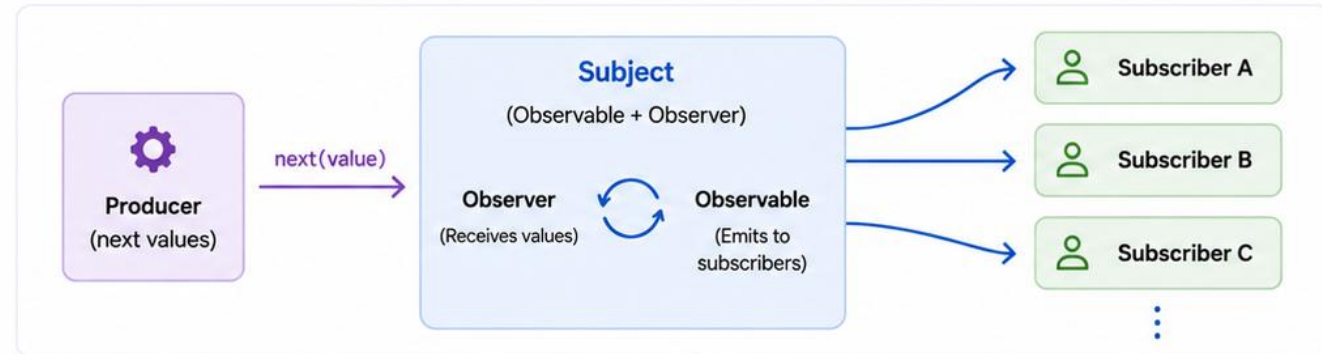


When to Use?

Use Subjects when you need to manually push values into a stream or share live data across multiple parts of your application.



Tip: Prefer BehaviorSubject for shared state (e.g., authentication, user data). Use ReplaySubject for caching or history. Use AsyncSubject for scenarios where only the final result matters (e.g., save operations).



☰ Types of Subjects



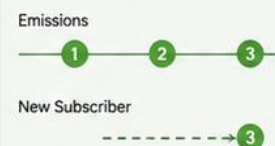
Subject

No initial value or buffer.
New subscribers receive only new values.



BehaviorSubject

Holds the latest value and emits it immediately to new subscribers.



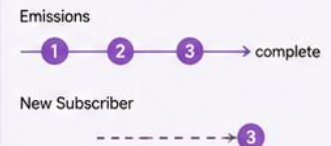
ReplaySubject

Replays a buffer of previous values (size or time based) to new subscribers.



AsyncSubject

Emits only the last value when the source completes.



```
import { Subject, BehaviorSubject, ReplaySubject, AsyncSubject } from 'rxjs';
```

BEHAVIORSUBJECT

A Subject with a memory: it always holds a current value and hands it to any new subscriber immediately.



Requires an Initial Value

`new BehaviorSubject<Customer[]>([])` -- unlike Subject, you must seed a starting value.



Late Subscribers Get the Current Value

Subscribe at any point and you immediately receive whatever was most recently pushed.



.value Property

Synchronously read the current value without subscribing -- useful, but use sparingly outside RxJS pipelines.

KEY TAKEAWAY BehaviorSubject is the RxJS type that actually behaves like state -- it always has a current value, which is exactly why it was Angular's go-to state container before signals existed.

TRY IT YOURSELF — EXERCISE 2: TRACE OBSERVABLES AND SUBJECTS

Reinforces: observable fundamentals, subscription, safe unsubscribe, Subject, and BehaviorSubject as you just walked them.

STEPS (notes/lab34-observables.md)

Step	What To Do
1 — Cold Start	Say what happens to an observable that is never subscribed to.
2 — Cleanup	Name two ways to avoid a leaking manual subscription.
3 — BehaviorSubject	Say what a late subscriber receives, and why that differs from Subject.
4 — Where It Fits	Name one place in the CRM UI where a stream beats a single value.

Stream behaviour

```
cold      : nothing runs until .subscribe() -- no subscriber, no work
cleanup   : async pipe, or takeUntilDestroyed()
Subject   : late subscriber gets nothing   BehaviorSubject: gets the current value
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-observable-basics.md (workspace: examples/module-34-exercises).



BehaviorSubject

A Subject that stores the latest value and emits it immediately to new subscribers.



Remembers Latest Value
Stores the most recent emission



Immediate Emission
New subscribers get the latest value right away



Hot Observable
Starts with an initial value



Great for State Management and Caching



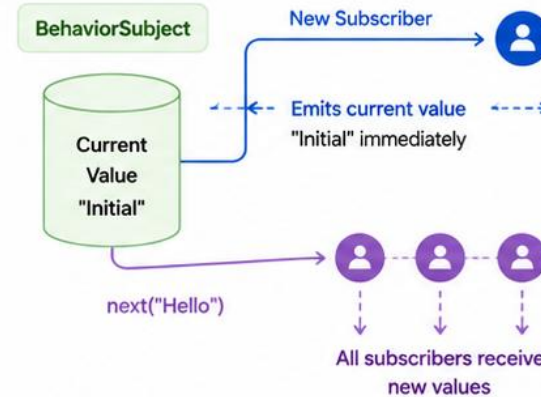
? What is BehaviorSubject?

BehaviorSubject is a type of Subject that **requires an initial value** and **always emits its current value** to any new subscriber. It is commonly used for state management and data sharing across components.

☰ Key Points

- Requires an initial value when created.
- Emits the current value to new subscribers immediately.
- Emits new values to all existing subscribers.
- Useful for state, configuration, and cached data.

How BehaviorSubject Works



```
import { BehaviorSubject } from 'rxjs';

// Create with an initial value
const subject = new BehaviorSubject<string>('Initial');

// First subscriber
subject.subscribe(value => console.log('Subscriber 1:', value));

subject.next('Hello'); // Subscriber 1: Hello
subject.next('World'); // Subscriber 1: World

// Second subscriber (gets latest value immediately)
subject.subscribe(value => console.log('Subscriber 2:', value));
// Subscriber 2: World (immediate emission)

subject.next('RxJS'); // Both subscribers: RxJS
```

🎯 Common Use Cases



Application State
Store and share application state (e.g., user info, theme, settings).



Configuration
Share configuration values that may change during runtime.



Caching
Cache API responses and provide the latest data to new subscribers.



Shared Services
Broadcast data changes from services to multiple components.

Example: Sharing User State

```
// user.service.ts
private userSubject = new BehaviorSubject<User | null>(null);
user$ = this.userSubject.asObservable();

setUser(user: User) {
  this.userSubject.next(user);
}

// component.ts
this.userService.user$.subscribe(user => {
  this.user = user; // gets latest user immediately
});
```

✅ Tips & Best Practices

- ✓ Always provide a meaningful initial value.
- ✓ Use BehaviorSubject for current state, not for events.
- ✓ Do not expose the subject directly; expose asObservable().
- ✓ Complete the subject when the service is destroyed.
- ✓ Prefer distinctUntilChanged() to avoid unnecessary emissions.



Key Takeaway: BehaviorSubject is perfect when new subscribers must always have the latest value. It combines the power of a Subject with the benefit of state retention.

SIGNALS VS OBSERVABLES

Two reactive primitives, overlapping in what they can represent but different in how they work and feel.

Aspect	Signals	Observables (RxJS)
Represents	A current value, synchronously readable	A stream of values arriving over time
Reading the value	customer() -- synchronous, always has a value	Must subscribe -- no synchronous 'current value' by default
Combining/transforming	computed() -- simple, limited operator set	A large, powerful library of operators (map, switchMap, debounceTime...)
Cleanup needed?	No -- signals have no subscriptions to leak	Yes -- subscriptions must be completed or unsubscribed
Best fit	Component/shared state, derived values	Genuinely async streams: HTTP, user-input timing, WebSockets

KEY TAKEAWAY Signals are simpler and safer for state you can read synchronously; RxJS is more powerful for genuinely time-based, asynchronous streams -- most real components use a bit of both.



Signals vs Observables

Two reactive primitives for different needs.



Signals

Synchronous, simple state management for UI.

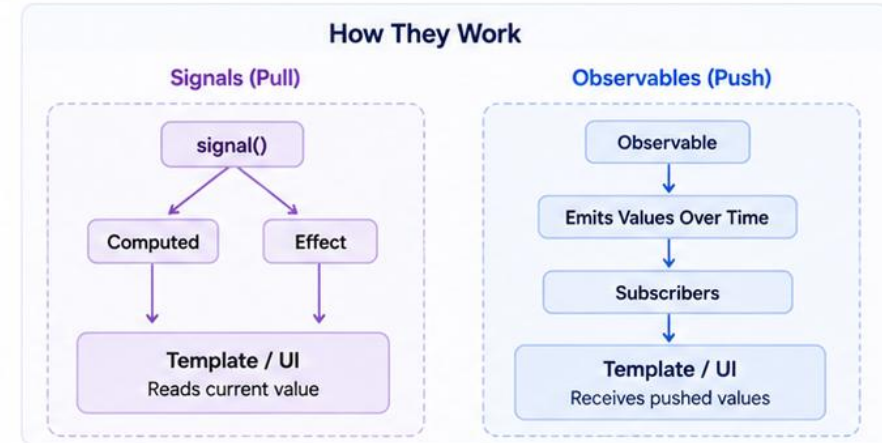


Observables

Asynchronous streams for events and I/O.



Aspect	Signals	Observables
Nature	Synchronous Values are read and updated synchronously.	Asynchronous Values are emitted over time, potentially asynchronously.
Data Model	Holds a single current value Always has the latest value.	Stream of values Can emit multiple values over time.
Push vs Pull	Pull-based Consumers pull the current value when needed.	Push-based Producers push values to subscribers when available.
Subscription	No subscriptions Consumers just read the value.	Requires subscription Must subscribe and unsubscribe (manually or via async pipe).
Memory Management	Automatic Angular tracks dependencies and cleans up automatically.	Manual (usually) Must unsubscribe to avoid memory leaks.
Best For	UI state, derived state, local component state e.g., form values, counters, toggles	Events, HTTP calls, WebSockets, intervals, complex async flows e.g., API calls, real-time updates
Example Use Cases	• Component state • Computed values • Derived UI state	• HTTP requests • WebSocket messages • Timers, intervals • Event streams
Angular Integration	Built-in (Angular 16+) Works natively with change detection and templates.	RxJS library Well-established, powerful reactive library.



Signals Example

```
import { signal, computed, effect } from '@angular/core';

const count = signal(0); // create signal

const double = computed(() => count() * 2); // derived

effect(() => {
  console.log('Count changed:', count());
});

count.set(1);
count.update(v => v + 1); // 2
console.log(double());    // 4
```

Observables Example

```
import { of, delay, map } from 'rxjs';

const source$ = of(1, 2, 3).pipe(
  delay(1000),
  map(v => v * 2)
);

const subscription = source$.subscribe({
  next: v => console.log('Value:', v),
  complete: () => console.log('Completed')
});

// Remember to unsubscribe if needed
subscription.unsubscribe();
```



Rule of Thumb

Use Signals for state that lives in memory and changes synchronously.
Use Observables for anything asynchronous or time-based.



Good to Know

You can convert between the two when needed:

- `toSignal(observable$)` → Observable to Signal
- `toObservable(signal)` → Signal to Observable



Key Takeaway

Signals simplify state management in Angular applications, while Observables remain essential for handling asynchronous data and event streams.

WHEN TO USE SIGNALS

Reach for a signal whenever the question is 'what is the current value of this,' synchronously.



Component & Shared State

Selected customer, filter text, loading flags -- anything a template reads directly.



Derived/Computed Values

Filtered lists, formatted strings, counts -- anything that is a pure function of other state.



Component Inputs

input()/input.required() are signals by design -- Module 33's modern @Input() replacement.

KEY TAKEAWAY If you find yourself asking 'what does this equal right now,' reach for a signal -- it is almost always the simpler, safer default.



When to Use Signals

Use Signals for synchronous, local, and UI state.
They simplify state management in Angular applications.



Synchronous
Values are available
immediately



Precise
Track exactly what
your UI needs



Performant
Built-in change
detection



Simple
Less boilerplate
and easier to use



What are Signals?

Signals are a **built-in Angular primitive** (Angular 16+) for managing reactive state. They **hold a value**, notify dependents automatically, and integrate seamlessly with the Angular change detection.



Key Characteristics

- ✓ **Synchronous** – values are read and updated immediately.
- ✓ **Fine-grained reactivity** – updates only what depends on it.
- ✓ **Easy to use** – simple API (signal, computed, effect).
- ✓ **Memory safe** – Angular tracks dependencies automatically.
- ✓ **Perfect for UI state** – designed for component and app-level state.



Use Signals For

- ✓ Component state (form fields, counters, toggles)
- ✓ UI state (loading, error, empty, dialogs, tabs)
- ✓ Derived state (computed values from other state)
- ✓ Shared state via services (theme, auth info, settings)
- ✓ Local caches and lightweight in-memory data
- ✓ Simple synchronous updates from user interactions



Ideal Use Cases for Signals



Component State

Manage local state within a component.

Examples:

Form values, counters, toggles, selected items.



UI State Management

Track UI-specific states that change often.

Examples:

Loading, error messages, empty states, dialogs, active tab.



Derived / Computed State

Compute values based on other signals.

Examples:

Totals, filtered lists, validation results, summary data.



Shared Application State

Share simple state across components using a service.

Examples:

User session, theme, language preference, feature flags.



Client-Side Caching

Store and reuse lightweight data in memory.

Examples:

Lookup data, user profile, configuration, dropdown options.



Event-Driven Updates

Update state in response to user actions.

Examples:

Button clicks, form submit, item selection, drag & drop.

Signals Are a Good Fit When...

- ✓ You need the current value right now.
- ✓ Updates happen synchronously.
- ✓ State is UI-centric and short-lived.
- ✓ You want simpler code with less boilerplate.



Signals Are Less Ideal When...

- ✗ You are dealing with asynchronous data streams.
- ✗ You need advanced stream operators.
- ✗ You are combining multiple async sources.
- ✗ You need cancellation, retry, debounce, etc.



Rules of Thumb

- ✓ Use Signals for synchronous, in-memory state.
- ✓ Use **computed()** for values derived from other signals.
- ✓ Use **effect()** for side effects (e.g., logging, localStorage sync).
- ✓ Prefer Signals over Observables for local UI state.
- ✓ Keep Signals simple and focused (single responsibility).
- ✓ Avoid using Signals for asynchronous streams (use Observables for that).



Example

For a todo app:

- ✓ Use Signals for the todo list, filters, UI flags, and selected todo.
- ✓ Use **Observables** for HTTP calls, WebSocket updates, or timers.



Key Takeaway: Use Signals for simple, synchronous, and UI-focused state. Reach for Observables when you need to handle asynchronous data and complex event streams.

WHEN TO USE RXJS

Reach for RxJS whenever timing, sequencing, or cancellation of async events is actually part of the problem.



Raw HTTP Calls

HttpClient returns Observables natively -- work with them directly, or convert to a signal with toSignal().



Debounced/Throttled Input

A live search box needs debounceTime and switchMap -- no signal-only equivalent exists.



Ongoing Streams

WebSocket messages, polling intervals, router events -- naturally continuous, not single-current-value state.

KEY TAKEAWAY If the problem genuinely involves 'when' and 'in what order' -- not just 'what is the value' -- RxJS's operators are doing real work that computed() cannot replace.

TRY IT YOURSELF — EXERCISE 3: DECIDE SIGNALS VERSUS RXJS

Reinforces: the Signals-versus-Observables comparison and the when-to-use-each slides you just covered, decided for real CRM cases.

STEPS (notes/lab34-signals-vs-rxjs.md)

Step	What To Do
1 — Three Cases	For the customer list, the search box, and a save button, pick one tool each.
2 — Justify	Give the deciding reason for each choice in one sentence.
3 — Interop	Say how an HTTP result becomes a signal the template can read.
4 — Team Rule	Write the one-line default your team follows when it is a toss-up.

Choosing a tool

```
current value, synchronous, drives the view    -> signal
events over time, debounce, cancel, retry      -> RxJS
http observable -> toSignal(...) -> read in the template
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-signals-vs-rxjs.md (workspace: examples/module-34-exercises).



When to Use RxJS

Use RxJS for asynchronous data streams, complex event handling, and powerful reactive compositions.



Asynchronous
Handle time-based and async data



Powerful Operators
Transform, filter, combine, and more



Multiple Emissions
Streams can emit zero, one, or many values



Cancellation
Unsubscribe, retry, debounce, throttle



Multicasting
Share a single source with many subscribers



What is RxJS?

RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using **Observables**.

It is designed to handle **asynchronous data streams** and **complex event-based** scenarios.



Key Characteristics

- ✓ **Asynchronous** – ideal for time-based and event-driven data.
- ✓ **Lazy** – nothing happens until you subscribe.
- ✓ **Multiple values** – emits zero, one, or many values over time.
- ✓ **Composability** – rich set of operators for transformation.
- ✓ **Cancellation** – unsubscribe, retry, debounce, throttle, etc.
- ✓ **Built for streams** – perfect for complex async workflows.



When RxJS is Less Ideal

- ✗ When you only need a single synchronous value
- ✗ For simple local component state
- ✗ When you don't need operators or stream composition
- ✗ When Signals or computed() can solve it easily

Use Signals for:

- ✓ Local UI state
- ✓ Computed values
- ✓ Component state
- ✓ Simple reactivity



RxJS vs Signals (Quick Guide)

Use Signals	Use RxJS
Synchronous, local UI state	Asynchronous, time-based, or event streams
Simple state updates	Complex logic with operators
Built-in, lightweight	External library, powerful
Best for component state	Best for services, HTTP, and global streams

Ideal Use Cases for RxJS



HTTP Requests

Handle API calls, loading states, errors, retries, and cancellations.

Examples:

GET/POST requests, polling, token refresh, request retries.



Real-Time Data

Work with WebSockets, Server-Sent Events, or live data streams.

Examples:

Live notifications, chat, stock prices, dashboards.



Time-Based Operations

Use timers, intervals, delays, and schedulers.

Examples:

Auto-save, countdowns, periodic refresh, animations.



User Input Streams

Handle continuous user events like typing, scrolling, resizing.

Examples:

Search with debounce, scroll events, drag & drop, resize.



Complex Compositions

Combine multiple streams and handle complex logic.

Examples:

CombineLatest, switchMap, forkJoin, merge, concat.



State Management

Manage app-wide state with Redux-like or RxJS-based stores.

Examples:

NgRx, Akita, custom stores.



Rules of Thumb

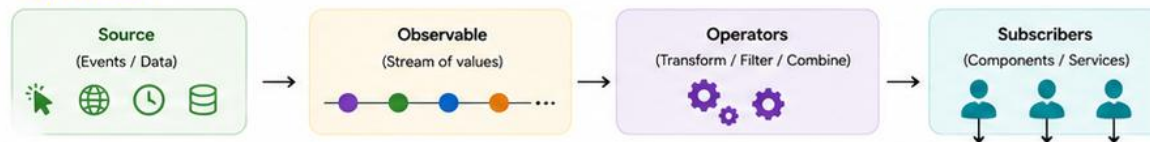
- ✓ Use RxJS when you are dealing with **streams over time**.
- ✓ Use RxJS for HTTP, WebSocket, timers, and complex async flows.
- ✓ Use **operators** to transform and control streams.
- ✓ Use **Subjects/BehaviorSubjects** to share and multicast.
- ✓ Use RxJS for **cross-component** or service-level communication.
- ✓ Avoid RxJS for simple local state – prefer Signals for that.



Example Scenarios

- Search box with debounce and API calls
- Paginated data loading with infinite scroll
- Live chat using WebSockets
- Auto-save form every few seconds
- Retry failed requests with exponential backoff
- Global loading indicator from multiple requests
- Route parameter changes triggering data reload

RxJS in Action



Key Takeaway

Use RxJS when you need to work with asynchronous data, time, or events over time and require **powerful stream transformations**.

If it's a stream, RxJS is the right tool.



Popular RxJS Operators

map	Transform values	debounceTime	Wait for silence
filter	Filter values	throttleTime	Limit frequency
switchMap	Cancel previous, switch to new	take / takeUntil	Complete / Unsubscribe
mergeMap	Merge all inner streams	catchError	Handle errors

EVENT BINDING

A quick review: Angular's (event)="handler()" syntax is the entry point for nearly everything this module covers.



DOM Events

(click), (input), (keyup.enter) -- native browser events bound directly to a class method.



Component Events

(selected) on a custom component -- the same syntax, listening to an @Output()/output().



The Trigger for Reactivity

An event handler is almost always where a signal .set()/update() or an RxJS .next() actually happens.

KEY TAKEAWAY Every signal update and every Subject.next() in this module has to start somewhere -- and that somewhere is almost always an event binding.



Event Binding

Event binding allows your Angular application to respond to user actions and DOM events.



Listen to Events
Capture user and DOM events.



Respond Dynamically
Call component methods when events occur.



Two-Way Interaction
Create interactive and reactive UIs.



User Driven
Build experiences that react to users.



What is Event Binding?

Event binding in Angular allows you to listen for events (like clicks, key presses, form submissions) and execute component logic in response.



Syntax

```
(event)="handlerMethod($event)"
```

(event) : The event to listen for (e.g., click, keyup)
handlerMethod() : The component method to call
\$event : The event object (optional)



Common Events

- (click) – Mouse click
- (dblclick) – Double click
- (keyup) – Key released
- (keydown) – Key pressed
- (input) – Input value changed
- (submit) – Form submitted
- (change) – Value changed
- (mouseover) – Mouse over an element
- (mouseleave) – Mouse leaves an element



Key Takeaways

- ✓ Event binding connects the UI with component logic.
- ✓ Use parentheses () to bind to events.
- ✓ \$event provides access to the event details.
- ✓ Essential for building interactive and dynamic applications.

Example: Button Click

Template (HTML)

```
<button class="btn btn-primary"
  (click)="onButtonClick($event)">
  Click Me
</button>

<p>You clicked {{ clickCount }} times.</p>
```

Component (TypeScript)

```
export class DemoComponent {
  clickCount = 0;

  onButtonClick(event: MouseEvent): void {
    this.clickCount++;
    console.log('Button clicked!', event);
  }
}
```

Example: Input Keyup

Template (HTML)

```
<input type="text"
  placeholder="Type something..."
  (keyup)="onKeyUp($event)" />

<p>Last pressed key: {{ lastKey }}</p>
```

Component (TypeScript)

```
export class DemoComponent {
  lastKey: string = '';

  onKeyUp(event: KeyboardEvent): void {
    this.lastKey = event.key;
  }
}
```

How It Works



User clicks the button



Component method is executed

Event Object (\$event)

Provides details about the event.

Example (click event):

```
{
  "type": "click",
  "target": <button>,
  "clientX": 120,
  "clientY": 200
}
```

- Contains information such as event type, target element, coordinates, key, etc.
- Useful for advanced scenarios.



Best Practices

- ✓ Keep event handlers simple and concise.
- ✓ Delegate complex logic to component methods.
- ✓ Avoid heavy operations in the template.
- ✓ Use \$event only when needed.



Common Mistakes

- ✗ Forgetting parentheses: `click="onClick()"` (Incorrect)
- ✗ Calling methods directly: `(click)="onClick()"` instead of `(click)="onClick"`
- ✗ Doing heavy logic in templates.
- ✗ Not using \$event when you need event details.

Incorrect Example

```
<button (click)="onClick()">
  Click
</button>
<!-- Calls onClick() immediately -->
```

HANDLING USER EVENTS

Beyond the click -- Angular gives you the same binding syntax for keyboard, focus, and form events.



Keyboard Events

(keyup.enter), (keydown.escape) -- Angular's key-filter syntax targets a specific key without manual checks.



Focus & Blur

(focus) and (blur) -- useful for showing/hiding validation messages exactly when relevant.



The \$event Object

Every handler can accept \$event -- the native DOM event, or a custom payload for component outputs.

KEY TAKEAWAY Angular's key-filter syntax like (keyup.enter) replaces a whole if (event.key === 'Enter') check with zero extra code in your handler.

Handling User Events

User events are the primary way users interact with your application. Angular lets you listen for events in the template and handle them in the component.



Key Concepts

- **Event Binding:** Use **(event)** syntax to listen to DOM or component events.
- **Event Handler:** Call a component method when the event occurs.
- **Context:** Handlers run in the component class, so you can access and update state.
- **Data Passing:** Pass event data (**\$event**) to handlers when needed.



Best Practice

Keep event handlers simple.
Delegate complex logic to services or other methods.

Template (HTML)

```
<button (click)="onSave()">
  Save
</button>

<input type="text"
  (input)="onInputChange($event)">

<div (mouseover)="onHover()">
  Hover over me
</div>
```

Component (TypeScript)

```
export class UserComponent {
  name = '';

  onSave(): void {
    console.log('Save clicked');
    // handle save logic
  }

  onInputChange(event: Event): void {
    const input = event.target as HTMLInputElement;
    this.name = input.value;
    console.log('Input changed:', this.name);
  }

  onHover(): void {
    console.log('Mouse is over the element');
  }
}
```



How It Works



User Action
User clicks, types, hovers, etc.



Angular Event
Angular captures the event.



Component Handler
Component method executes.



Common Events

(click)	Button clicks, link clicks
(input)	User types into an input field
(change)	Value changes (select, checkbox, input)
(submit)	Form submission
(mouseover)	Mouse enters an element
(keydown)	Key pressed on keyboard



Remember: Events connect the user's actions to your application logic. Handle them cleanly and update state reactively.

CUSTOM COMPONENT EVENTS

A quick review, applied: how a child announces something happened, and how the parent's state actually reacts.



Emit From the Child

```
selected = output<Customer>(); ...  
this.selected.emit(this.customer());
```



Handle in the Parent

```
(selected)='onCustomerSelected($event)' --  
the parent's handler updates its own state.
```



State Update Follows

The handler almost always ends in a signal `.set()/.update()` call -- closing the loop back to this module's core topic.

KEY TAKEAWAY Custom events (from Module 33) and signal updates (from this module) are two halves of the same sentence -- a child emits, a parent reacts by updating state.



Custom Component Events

Custom component events let a child component notify its parent about actions or changes. Angular uses `@Output()` and `EventEmitter` to create and emit custom events.

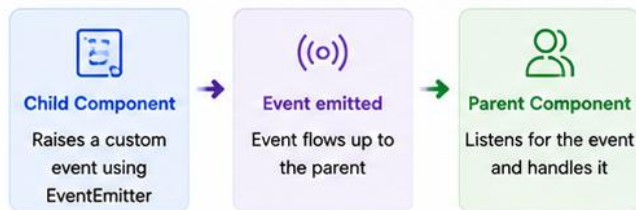


Why Use Custom Events?

- ✓ Enable child-to-parent communication
- ✓ Keep components reusable and decoupled
- ✓ Emit meaningful events (e.g., item selected, saved)
- ✓ Pass data along with the event



How It Works



Child Component (emits event)

```
child.component.ts
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  templateUrl: './child.component.html'
})
export class ChildComponent {
  @Output() itemSelected = new EventEmitter<string>();

  selectItem(item: string): void {
    this.itemSelected.emit(item); // emit event with data
  }
}
```

child.component.html

```
<button class="btn btn-primary" (click)="selectItem('Laptop')">
  Select Laptop
</button>

<button class="btn btn-primary" (click)="selectItem('Phone')">
  Select Phone
</button>
```

Parent Component (listens for event)

parent.component.html

```
<app-child (itemSelected)="onItemSelected($event)"></app-child>

<p class="mt-3">
  Selected Item: <strong>{{ selectedItem }}</strong>
</p>
```

parent.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  templateUrl: './parent.component.html'
})
export class ParentComponent {
  selectedItem: string = '';

  onItemSelected(item: string): void {
    this.selectedItem = item;
    console.log('Item selected:', item);
  }
}
```



Best Practice:

Name events with action or intent (e.g., `itemSelected`, `formSubmitted`) and always emit meaningful data.



Event Binding Syntax

```
<child-component (eventName)="handlerMethod($event)"> </child-component>
```

`$event` contains the data passed from the child component.



Key Takeaway: Use `@Output()` and `EventEmitter` to create custom events. Emit data from child components and handle it in the parent using event binding.

PARENT-CHILD EVENT FLOW

Trace one interaction end to end -- click, emit, handle, update, re-render -- as a single connected chain.

1 User Clicks a Card

A (click) handler inside CustomerCardComponent fires.

2 Card Emits 'selected'

`this.selected.emit(this.customer( ))` sends the Customer up to whoever is listening.

3 Parent Updates State

`onCustomerSelected(customer)` calls `this.selectedCustomer.set(customer)` in the list page.

4 Template Re-Renders

Any binding reading `selectedCustomer()` -- a detail panel, a highlighted row -- updates automatically.

KEY TAKEAWAY This four-step chain is the complete answer to 'how does clicking something in Angular actually change what's on screen' -- memorize the shape, not just the code.

TRY IT YOURSELF — EXERCISE 4: TRACE EVENT FLOW UPWARD

Reinforces: event binding, user event handling, custom component events, and parent-child event flow as you just saw them.

STEPS (notes/lab34-event-flow.md)

Step	What To Do
1 — Event Chain	Trace a row click from the template to the signal that changes.
2 — Custom Event	Define the child's output and the type its payload carries.
3 — Who Owns State	Say which component updates the signal, and why not the child.
4 — Two Levels Up	Say what happens when a grandchild needs to notify a grandparent.

Event chain

```
(click) in row -> emit selected(customer) -> parent handler
  -> selectedId.set(customer.id) -> computed view updates
grandchild: re-emit at each level, or share a service signal
```

TRY IT NOW — Complete Exercise 4 before continuing: `exercises/exercise-04-event-flow.md` (workspace: `examples/module-34-exercises`).



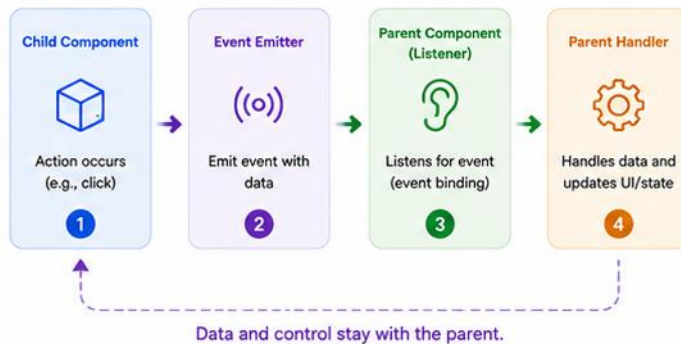
Parent–Child Event Flow

Events flow from child to parent. The child component emits an event, and the parent component listens for it and handles the data.

How It Works

- 1 Child component triggers an action (e.g., button click).
- 2 Child emits a custom event with data using EventEmitter.
- 3 Parent listens for the event using (eventName) binding.
- 4 Parent handles the event and updates its state or UI.

Event Flow Diagram



Child Component (Emitter)

```
child.component.ts
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  templateUrl: './child.component.html'
})
export class ChildComponent {
  @Output() itemSelected = new EventEmitter<string>();

  selectItem(item: string): void {
    console.log('Child: item selected ->', item);
    this.itemSelected.emit(item); // Emit event with data
  }
}
```

```
child.component.html
<div class="child">
  <h4>Child Component</h4>
  <ul>
    <li *ngFor="let item of items">
      <button (click)="selectItem(item)->{{ item }}"></button>
    </li>
  </ul>
</div>
```

Key Point

The child does not modify the parent directly. It only emits an event with data.

Parent Component (Listener)

```
parent.component.html
<app-child (itemSelected)="onItemSelected($event)"></app-child>

<div class="parent">
  <h4>Parent Component</h4>
  <p>Selected Item: <strong>{{ selectedItem }}</strong></p>
</div>
```

```
parent.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  templateUrl: './parent.component.html'
})
export class ParentComponent {
  selectedItem: string = '';

  onItemSelected(item: string): void {
    console.log('Parent: received item ->', item);
    this.selectedItem = item; // Handle event data
    // Update UI, call service, etc.
  }
}
```

Best Practice

Keep components reusable and communicate via events or inputs. Avoid tight coupling between parent and child.

Key Takeaway: Use custom events to notify parents about changes or user actions in child components.

Child Action → Emit Event → Parent Listens → Handle & Update

FORM STATE OVERVIEW

A form is not just markup -- it is a bundle of state: values, validity, and interaction history, per field and as a whole.



Value

What the user has actually typed or selected right now, per field and for the form overall.



Validity

Does this value satisfy its rules -- required, min length, a valid email pattern?



Interaction History

Has this field been touched (focused then left) or is it still pristine (never interacted with)?

KEY TAKEAWAY Every form-handling decision in Angular ultimately comes down to reading or reacting to one of these three kinds of state.



Form State Overview

Form state represents the current values, validation status, and interaction state of user inputs. Angular tracks form state to help you build robust and user-friendly forms.

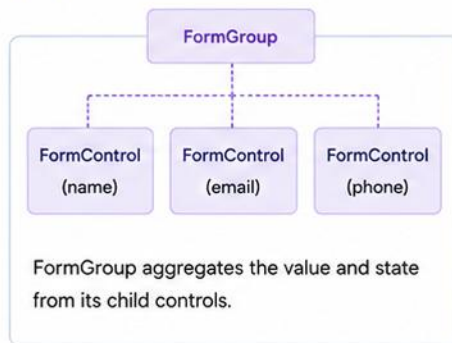


Key Concepts

- Value State:** Current value(s) of the form or form controls.
- Validation State:** Whether the form is valid or invalid based on validation rules.
- Interaction State:** Tracks how the user interacts with the form (touched, dirty, etc.).
- Aggregate State:** Parent form groups aggregate state from their child controls.



Form State Model



Form State Properties

Property	Type	Description
value	any	Current value of the control or form.
valid	boolean	True if all validations pass.
invalid	boolean	True if any validation fails.
dirty	boolean	True if the user changed the value.
pristine	boolean	True if the value has not been changed.
touched	boolean	True if the control has been blurred.
untouched	boolean	True if the control has not been blurred.
pending	boolean	True if async validation is in progress.
errors	ValidationErrors null	Contains validation errors, if any.
status	string	One of: VALID , INVALID , PENDING , DISABLED



State Lifecycle Flow



Example: Reactive Form State

Component (TypeScript)

```
this.form = this.fb.group({
  name: ['', [Validators.required, Validators.minLength(2)]],
  email: ['', [Validators.required, Validators.email]],
  phone: ['', [Validators.pattern(/^[0-9]{10}$/)]]
});

// Accessing state
console.log(this.form.value);
console.log(this.form.valid);
console.log(this.form.dirty);
console.log(this.form.touched);
console.log(this.form.get('email')?.errors);
```

Template (HTML)

```
<form [formGroup]="form" novalidate>
  <input formControlName="name" placeholder="Name" />
  <input formControlName="email" placeholder="Email" />
  <input formControlName="phone" placeholder="Phone" />

  <button type="submit" [disabled]="form.invalid">Submit</button>
</form>
```



Common States

Pristine + Untouched	User hasn't interacted yet.
Dirty + Touched	User changed value and left the field.
Invalid	One or more validation rules failed.
Valid	All validation rules passed.
Pending	Async validation is in progress.
Disabled	Control is disabled (excluded from validation).



Best Practice: Keep validation rules simple, show errors only after the field is touched or the form is submitted, and always provide clear feedback.

TEMPLATE-DRIVEN FORMS

Form state lives mostly in the template itself, driven by ngModel and the FormsModule directives.



FormsModule Required

Import FormsModule to unlock ngModel and its related directives in a standalone component.



[(ngModel)] Two-Way Binding

```
<input [(ngModel)]= 'customer.email'
name='email' /> -- the form field and a class
property stay in sync.
```



Best For Simple Forms

A short contact form or a single search field -
- where explicit, class-side validation logic is overkill.

KEY TAKEAWAY Template-Driven Forms feel the fastest to write for a small form -- but that convenience trades away the explicit, class-side control Reactive Forms give you.



Template-Driven Forms

Template-driven forms use Angular directives in the template to create and manage form controls. They are simple to build and ideal for straightforward forms with minimal dynamic behavior.



Key Concepts

- Driven by the template using directives.
- Angular creates `FormControl` instances behind the scenes.
- Uses `NgForm` to track the overall form state.
- Ideal for simple forms with minimal logic.
- Easy to validate and display errors.



How It Works

- Add template-driven directives.
- Angular creates form controls automatically.
- `NgForm` tracks the form state.
- Use the form state in the template.
- Handle submit using `(ngSubmit)`.



Best Practice

- Use for simple forms.
- Use template variables (e.g., `#name="ngModel"`).
- Show errors only after control is touched or form submitted.



Example: Template-Driven Form for User Registration

Template (HTML)

```
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
  <div class="form-group">
    <label for="name">Name</label>
    <input id="name" name="name" [(ngModel)]="user.name"
      required #name="ngModel" class="form-control" />
    <div class="text-danger" *ngIf="name.invalid && name.touched">
      Name is required
    </div>
  </div>
  <div class="form-group">
    <label for="email">Email</label>
    <input id="email" name="email" type="email" [(ngModel)]="user.email"
      required email #email="ngModel" class="form-control" />
    <div class="text-danger" *ngIf="email.invalid && email.touched">
      Valid email is required
    </div>
  </div>
  <button type="submit" [disabled]="userForm.invalid" class="btn btn-primary">
    Register
  </button>
</form>
```

Component (TypeScript)

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-register',
  templateUrl: './register.component.html'
})
export class RegisterComponent {
  user = { name: '', email: '' };
  onSubmit(form: NgForm) {
    if (form.valid) {
      console.log('Form submitted: ', this.user);
      // call API or service
    }
  }
}
```



Notes

- `ngModel`: Binds the input value to a component property.
- `name`: Required for `ngModel` to register the control.
- `#name="ngModel"`: Template reference to access control state.
- `NgForm`: Tracks the whole form state.



Form State with NgForm

Property	Description
valid	True if all controls are valid.
invalid	True if one or more controls are invalid.
dirty	True if any control value has changed.
pristine	True if no control value has changed.
touched	True if any control has been visited.
untouched	True if no control has been visited.
submitted	True if the form has been submitted.
controls	Collection of all form controls.



Template-Driven vs Reactive Forms

Feature	Template-Driven	Reactive
Configuration	In template	In component
Best for	Simple forms	Complex, dynamic forms
Setup	Quick and easy	More setup
Validation	Template directives	Programmatic
Control access	Template refs	<code>FormControl</code> instances
Flexibility	Lower	Higher

Common Directives

- `ngModel`
- `NgForm`
- `required`
- `email`, `minlength`, `maxlength`
- `ngModelOptions`
- `(ngSubmit)`



Key Takeaway: Template-driven forms are easy to build and perfect for simple forms. Use them when your form logic is straightforward.

REACTIVE FORMS

Form state lives explicitly in the component class, as FormControl/FormGroup objects you construct in code.



ReactiveFormsModule Required

Import ReactiveFormsModule to unlock FormControlName, FormGroup, and the class-side FormGroup API.



Built in the Class

```
customerForm = new FormGroup({ email:  
  new FormControl('') }); -- explicit, typed,  
  testable without a template.
```



Unit-Testable Validation

Because the form lives in the class, you can assert its validity in a plain unit test -- no DOM required.

KEY TAKEAWAY Reactive Forms trade a bit more setup code for explicit, typed, class-side control -- the right default for crm-ui's real forms with real validation rules.



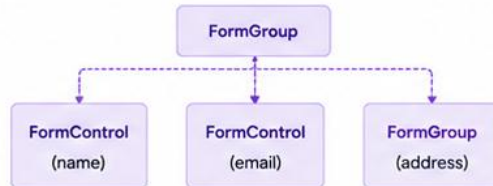
Reactive Forms

Reactive forms in Angular use an explicit, immutable approach to build and manage form state. They provide more control, validation, and scalability for complex, dynamic forms.

Key Concepts

- Model-driven:** Form model is created in the component (TypeScript).
- Immutable:** Update values through the form API.
- Observable:** Form state and value changes are observable.
- More powerful validation:** Synchronous and asynchronous.
- Best for complex or dynamic forms.**

Form Model Structure



Common Use Cases

Complex forms with multiple fields
Dynamic form controls (add/remove at runtime)
Advanced validation scenarios
Cross-field validation
Large enterprise applications

Example: Reactive Form for User Registration

```

Component (TypeScript)
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-register',
  templateUrl: './register.component.html'
})
export class RegisterComponent implements OnInit {
  registerForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    this.registerForm = this.fb.group({
      name: ['', [Validators.required, Validators.minLength(3)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]],
      address: this.fb.group({
        street: ['', Validators.required],
        city: ['', Validators.required],
        zip: ['', [Validators.required, Validators.pattern(/^\d{0-9}{5,6}$/)]]
      })
    });
  }

  onSubmit(): void {
    if (this.registerForm.valid) {
      console.log('Form submitted:', this.registerForm.value);
      // call API or service
    } else {
      this.registerForm.markAllAsTouched();
    }
  }
}
  
```

Template (HTML)

```

<form [formGroup]="registerForm" (ngSubmit)="onSubmit()">
  <div class="form-group">
    <label for="name">Name</label>
    <input id="name" formControlName="name" />
    <div class="text-danger" *ngIf="registerForm.get('name')?.touched &&
      registerForm.get('name')?.invalid">
      Name is required (min 3 characters)
    </div>
  </div>
  <div class="form-group">
    <label for="email">Email</label>
    <input id="email" type="email" formControlName="email" />
    <div class="text-danger" *ngIf="registerForm.get('email')?.touched &&
      registerForm.get('email')?.invalid">
      Valid email is required
    </div>
  </div>
  <div formGroupName="address" class="form-group">
    <label for="street">Street</label>
    <input id="street" formControlName="street" />
    <label for="city">City</label>
    <input id="city" formControlName="city" />
    <label for="zip">ZIP</label>
    <input id="zip" formControlName="zip" />
  </div>
  <button type="submit" [disabled]="registerForm.invalid">Register</button>
</form>
  
```

Reactive Form Flow



Validation in Reactive Forms

- Built-in Validators:** required, email, minLength, pattern, etc.
- Custom Validators:** Create synchronous or async validators.
- Cross-field Validation:** Validate based on multiple fields.

Best Practice: Use reactive forms for complex, dynamic, or large-scale forms where maintainability and testability are critical.

FORMCONTROL

One FormControl instance tracks everything about a single form field -- its value, validity, and touch state together.

Property / Method	What It Gives You
<code>.value</code>	The field's current value -- <code>email.value</code> reads it synchronously.
<code>.valid</code> / <code>.invalid</code>	Whether the current value satisfies every validator attached to this control.
<code>.dirty</code> / <code>.pristine</code>	Whether the value has ever been changed from its initial value.
<code>.touched</code> / <code>.untouched</code>	Whether the field has been focused and then blurred at least once.
<code>.setValue(v)</code> / <code>.patchValue(v)</code>	Programmatically set the field's value from code, e.g. when loading an existing customer to edit.

KEY TAKEAWAY A FormControl is the atomic unit of Reactive Forms -- every field in `customerForm` is one, and `FormGroup` is simply a named collection of them.



FormControl

FormControl is the fundamental building block of reactive forms. It tracks the value, validation state, and interaction state of a single form input.



Key Concepts

- Represents the value and state of a single form input.
- Tracks value, validation status, and interaction state.
- Immutable updates through the form API.
- Supports synchronous and asynchronous validators.
- Can be used standalone or inside a FormGroup / FormArray.



Creating a FormControl

```
import { FormControl, Validators } from '@angular/forms';

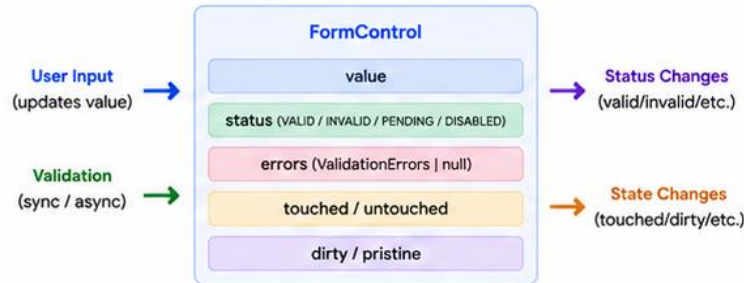
// Simple control
const nameCtrl = new FormControl('Alice');

// With validators
const emailCtrl = new FormControl(
  '',
  [Validators.required, Validators.email]
);

// With initial value and disabled state
const ageCtrl = new FormControl({
  value: 25,
  disabled: true
});
```



FormControl Anatomy



TypeScript Example

```
import { FormControl, Validators } from '@angular/forms';
const email = new FormControl('', [Validators.required, Validators.email]);
email.valueChanges.subscribe(value => console.log('Value:', value));
email.statusChanges.subscribe(status => console.log('Status:', status));
```



Status Values

Status	Description
VALID	All validators passed.
INVALID	One or more validators failed.
PENDING	Async validation is in progress.
DISABLED	Control is disabled; excluded from validation.



Using FormControl in Template

```
<form [formGroup]="userForm" (ngSubmit)="onSubmit()">
  <div>
    <label for="name">Name</label>
    <input id="name" type="text" formControlName="name" />
    <div class="error" *ngIf="name.invalid && name.touched">
      Name is required (min 3 characters)
    </div>
  </div>
</form>
```



Note:

FormControl is usually accessed via formControlName when used inside a FormGroup.



Common Validators

Validator	Purpose
Validators.required	Value must be present.
Validators.email	Value must be a valid email.
Validators.minLength(n)	Minimum length of n.
Validators.maxLength(n)	Maximum length of n.
Validators.pattern(regex)	Value must match the pattern.
Validators.min(n)	Minimum numeric value.
Validators.max(n)	Maximum numeric value.
Validators.nullValidator	No validation (obsolete).



Best Practice

Keep controls small and focused. Apply validators to the control level and use FormGroup for grouping related controls.



Goal:

Build predictable, maintainable, and scalable forms with precise control over state and validation.

FORMGROUP

A FormGroup bundles multiple FormControl(s) (or nested FormGroups) into one object with aggregated state.



A Named Collection

`new FormGroup({ name: new FormControl(""),
email: new FormControl("") })` -- keys become
field names.



Aggregated Validity

The FormGroup is `.valid` only when every one
of its controls is individually `.valid`.



Nested Groups

A FormGroup can contain another
FormGroup -- e.g. a nested 'address' section
within a customer form.

KEY TAKEAWAY `customerForm.valid` is true exactly when every individual FormControl inside it is valid -- FormGroup does the aggregation so you never manually AND together each field's validity yourself.



FormGroup

FormGroup is a container that holds and manages a group of FormControl, FormGroup, or FormArray instances as a single unit.



Key Concepts

- Groups multiple form controls into a single form model.
- Tracks the combined value and status of all child controls.
- Simplifies form validation and state management.
- Supports nested groups and arrays.
- Commonly used for structured forms (e.g., user registration form).



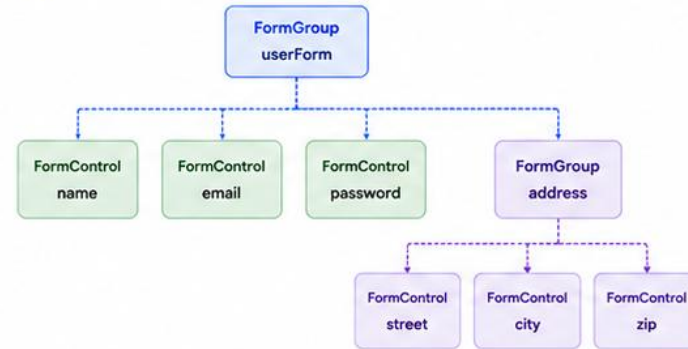
Creating a FormGroup

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

const userForm = new FormGroup({
  name: new FormControl('', [Validators.required, Validators.minLength(3)]),
  email: new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', [Validators.required, Validators.minLength(6)]),
  address: new FormGroup({
    street: new FormControl('', Validators.required),
    city: new FormControl('', Validators.required),
    zip: new FormControl('', [Validators.required, Validators.pattern(/^\d{0-9}{5,6}$/)])
  })
});
```



FormGroup Structure



Value Example (userForm.value)

```
{
  "name": "Alice Smith",
  "email": "alice@example.com",
  "password": "password123",
  "address": {
    "street": "123 Main St",
    "city": "Brampton",
    "zip": "L6R1P3"
  }
}
```



Status Aggregation

- **VALID:** All child controls are valid.
- **INVALID:** One or more child controls are invalid.
- **PENDING:** Any child control has async validation in progress.
- **DISABLED:** All child controls are disabled.



Using FormGroup in Template

```
<form [formGroup]="userForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Name</label>
    <input formControlName="name" />
    <div *ngIf="userForm.get('name')?.invalid &&
      userForm.get('name')?.touched" class="error">
      Name is required (min 3 characters)
    </div>
  </div>
  <div formGroupName="address">
    <label>Street</label>
    <input formControlName="street" />
    <label>City</label>
    <input formControlName="city" />
    <label>Zip</label>
    <input formControlName="zip" />
  </div>
  <button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>
```



Common FormGroup APIs

API	Description	Example
<code>get(path)</code>	Get a child control by path	<code>userForm.get('name')</code>
<code>controls</code>	Access all child controls	<code>userForm.controls</code>
<code>value</code>	Get the combined value	<code>userForm.value</code>
<code>status</code>	Get the combined status	<code>userForm.status</code>
<code>patchValue()</code>	Update part of the value	<code>userForm.patchValue({ name: 'Alice' })</code>
<code>reset()</code>	Reset the form value and state	<code>userForm.reset()</code>
<code>setValidators()</code>	Set validators on the group	<code>userForm.setValidators(...)</code>



Best Practices

- ✓ Keep FormGroups focused and manageable.
- ✓ Use meaningful names for controls and groups.
- ✓ Apply validators at the group level for cross-field validation.
- ✓ Use FormGroup for logical sections of a form.



When to Use

- When your form has multiple related fields.
- When you need group-level validation.
- When building complex or nested forms.
- When managing form state as a single unit.

FORM VALIDATION

Validators are plain functions attached to a FormControl that decide, on every value change, whether it passes.

Validator	Checks
Validators.required	The value is not empty, null, or undefined.
Validators.minLength(n) / maxLength(n)	A string's length is within the given bounds.
Validators.pattern(regex)	The value matches a given regular expression.
Validators.email	The value looks like a syntactically valid email address.
A custom validator function	Any (control: AbstractControl) => ValidationErrors null your logic needs.

KEY TAKEAWAY Built-in validators cover most real form rules; a custom validator function is just a plain function returning an error object or null -- no different in kind from the built-ins.

TRY IT YOURSELF — EXERCISE 5: PLAN THE REACTIVE FILTER FORM

Reinforces: reactive forms, FormControl, FormGroup, and validation as you just covered them, planned for the CRM filter and edit form.

STEPS (notes/lab34-form-plan.md)

Step	What To Do
1 — Controls	Name the FormGroup and its controls for filtering by name and status.
2 — Validation	Give one validator and the message the user sees when it fails.
3 — Wire to State	Say how a value change reaches the filter signal from Exercise 1.
4 — Reactive vs Template	Give the reason this cohort uses reactive forms here.

Filter form sketch

```
filterForm = new FormGroup({ name: new FormControl(''),
                             status: new FormControl('ALL') })
valueChanges -> debounceTime(300) -> filter.set(value)
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-reactive-form-plan.md (workspace: examples/module-34-exercises).



Form Validation

Ensure data integrity and a great user experience with client-side validation.



Why Validate?

Validation improves data quality, prevents errors, and provides immediate feedback to users before submitting data to the server.



Common Validation Types

- ✓ Required – Field cannot be empty
- ✓ Min/Max Length – Enforce length constraints
- ✓ Pattern – Match with regular expressions
- ✓ Min/Max Value – Numeric range validation
- ✓ Email, URL, Date – Built-in format validators
- ✓ Custom Validators – Business rule validation

Employee Registration

Full Name *

John Doe

Looks good!

Email *

john.doe@

Please enter a valid email address.

Password *

•••••

Password must be at least 8 characters long.

Phone Number

(555) 123-4567

Department *

Engineering

Great choice!

Submit



All required fields must be valid

Validation Flow



1. User Input

User enters data in the form fields.



2. Validate

Validators check the value against defined rules.



3. Feedback

Show error messages or success indicators.



4. Submit

Form is valid → data is submitted to the backend.



Form Validation Best Practices



Validate early and provide clear, actionable messages.



Use accessible error messages and ARIA roles.



Validate on the client and again on the server.



Keep validation logic simple and maintainable.



Good validation = Better data quality + Better user experience + Fewer production issues

</> Angular Reactive Forms • Template-Driven Forms • Built-in & Custom Validators

UI REACTIVITY

Everything covered so far -- signals, events, forms -- exists to answer one question: does the screen reflect reality?



The Screen Follows State

Every pixel a user sees should trace back to a signal or an async-piped Observable -- never a stale one-time render.



Reactivity Is Automatic

Once wired correctly, you never manually call 'update the DOM' -- Angular does it the instant tracked state changes.



Breakage Looks Like Silence

A reactivity bug rarely throws an error -- it just quietly stops updating something.

KEY TAKEAWAY A genuinely reactive UI needs no manual refresh logic anywhere -- if you ever find yourself calling something like `this.reload()`, that is usually a sign a piece of state is not actually wired reactively.



UI Reactivity

Keep the UI in sync with data changes automatically.



What is UI Reactivity?

UI reactivity ensures the user interface updates automatically whenever the underlying data changes — without manual DOM manipulation.



Core Principles

- ✓ **Declarative** – Describe what the UI should look like, not how to update it.
- ✓ **Automatic** – Changes in state propagate to the UI automatically.
- ✓ **Efficient** – Only the parts of the UI that change are updated.
- ✓ **Consistent** – UI always reflects the current state.

Reactive Data Flow



Example: Employee List

State (Signal)

```
const employees = signal<Employee[]>([  
  { id: 1, name: 'Alice', role: 'Developer' },  
  { id: 2, name: 'Bob', role: 'Designer' }  
]);
```

UI (Template)

Employees			Add Employee
1	Alice	Developer	
2	Bob	Designer	

When State Changes...

```
employees.update(list => [...list,  
  { id: 3, name: 'Charlie', role: 'QA' }]);
```

Employees			Add Employee
1	Alice	Developer	
2	Bob	Designer	
3	Charlie	QA	NEW



The UI stays in sync automatically!
No manual DOM updates. No setTimeout. No refresh calls.

Reactivity Approaches in Angular



Signals (Angular)

- Fine-grained reactivity
- Synchronous updates
- Lightweight and performant



RxJS Observables

- Stream-based reactivity
- Ideal for async data (HTTP, WebSocket)
- Powerful operators

Benefits of UI Reactivity



Better User Experience

UI updates instantly, leading to a smooth and responsive experience.



Improved Performance

Angular updates only what changed, reducing unnecessary re-renders.



Fewer Bugs

Less manual DOM code means fewer inconsistencies and errors.



Cleaner Code

Declarative patterns make components easier to read and maintain.



UI Reactivity Best Practices



Keep state minimal
Store only what's necessary.



Use immutable updates
Helps Angular detect changes efficiently.



Use OnPush change detection
Optimize component change detection.



Unsubscribe from Observables
Prevent memory leaks in components.



Profile & Optimize
Measure performance and fine-tune regularly.



Reactive UI = Happy Users + Maintainable Code + High Performance

Signals • Observables • Change Detection • Components

ASYNC DATA AND STATE

Real data does not arrive instantly -- every async operation has at least three states your UI must represent.



In Flight

The request has started but not resolved yet -
- the UI should say so, not just look frozen.



Resolved

Data arrived successfully -- render it.



Failed

The request errored -- the UI needs an honest, actionable message, not a silent blank screen.

KEY TAKEAWAY Model every async operation as a small state machine -- loading, success, or error -- rather than just a variable that eventually gets a value.



Async Data and State

Handle asynchronous data effectively
and keep the UI responsive and predictable.



Why Async State Matters

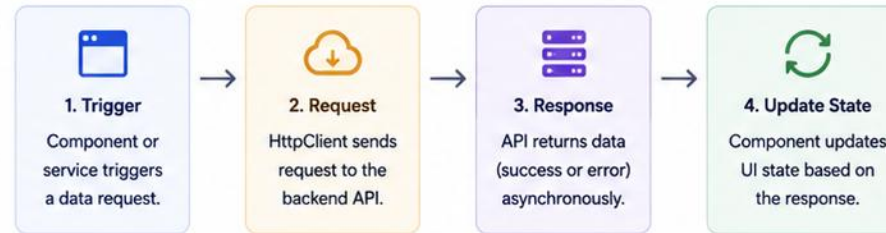
Data from APIs, databases, or services is not available immediately. Managing async state provides a clear experience for users in all situations.



Key Considerations

- ✓ Data is fetched asynchronously.
- ✓ UI must reflect loading, success, error, and empty states.
- ✓ Avoid memory leaks and unnecessary subscriptions.
- ✓ Keep state transitions predictable.

Async Data Flow



Example: Angular Service (TypeScript)

```
getEmployees(): Observable<Employees[]> {  
  return this.http.get<Employees[]>(`${environment.apiUrl}/employees`)  
    .pipe(  
      map(res => res),  
      catchError(err => {  
        console.error('API Error', err);  
        return throwError(() => err);  
      })  
    );  
}
```

Example: Component State

Employees Refresh

Loading...
Fetching employee data.

Data Loaded
120 employees found.

No Data
No employees available.

Error
Failed to load employees. Retry

Common Async States



Loading

Request is in progress.
Show spinners or skeleton loaders.



Success

Data is loaded successfully.
Render the UI with data.



Empty

Request succeeded but no data.
Show empty state UI.



Error

Request failed.
Show error message and retry option.

Benefits



Better User Experience

Users always know what's happening.



Resilient UI

Handles failures and edge cases gracefully.



Predictable State Transitions

Easier to test and maintain.



Async State Best Practices



Use RxJS & Observables
Leverage RxJS for composable and cancelable async flows.



Unsubscribe Properly
Use async pipe or takeUntil to prevent memory leaks.



Handle All States
Always implement loading, success, empty, and error states.



Keep UI Responsive
Avoid blocking operations in the main thread.



Centralize Error Handling
Use Interceptors and global error strategies.



Good async state management = Better UX + Fewer bugs + Easier maintenance



HttpClient



Observables



State



UI

LOADING, ERROR, AND EMPTY STATE

Three UI states, each needing its own honest representation -- none of them is just 'the default view with less data.'



Loading State

`isLoading()` drives a spinner or skeleton -- shown the instant a request starts, hidden the instant it settles.



Error State

`error()` holds a user-facing message, set in the `.subscribe()` error callback -- never leave a failed request silent.



Empty State

`customers().length === 0` after a successful load is not an error -- it needs its own message, distinct from both loading and failure.

KEY TAKEAWAY Loading, error, and empty are three genuinely different situations that all look like 'no customer rows' if you do not model them separately -- and each needs its own honest message.



Loading State

Indicate that data is being fetched and the UI is in progress.



Why It Matters

- Sets user expectations
- Prevents confusion and duplicate actions
- Improves perceived performance
- Provides a smoother user experience



Best Practices

- ✓ Show loading immediately when a request starts
- ✓ Keep the UI responsive
- ✓ Use spinners, skeletons, or progress bars
- ✓ Avoid blocking the entire page
- ✓ Hide loader as soon as data is ready or fails

Loading State Flow



Example: Angular Component (TypeScript)

```
export class EmployeeListComponent {
  employees: Employee[] = [];
  loading = false;
  error: string | null = null;

  constructor(private service: EmployeeService) {}

  loadEmployees() {
    this.loading = true;
    this.error = null;
    this.service.getEmployees().subscribe({
      next: (data) => {
        this.employees = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Failed to load employees.';
        this.loading = false;
      }
    });
  }
}
```

Example: UI States

Loading...

ID	Name	Role
1	Alice Johnson	Developer
2	Bob Smith	Designer
3	Charlie Brown	QA Engineer

Loading employees... Please wait.

Data Loaded

ID	Name	Role
1	Alice Johnson	Developer
2	Bob Smith	Designer
3	Charlie Brown	QA Engineer

120 employees loaded successfully.

Error

Failed to load employees. Please try again. [Retry](#)

Common Loader Patterns



Spinner

Simple and lightweight. Great for small areas.



Dots / Pulse

Subtle animations that show activity without distraction.



Progress Bar

Use when progress is known or takes longer.



Skeleton UI

Shows content structure while data is loading.



Overlay Loader

Use for full-page or modal blocking operations.



Pro Tip

Prefer skeletons for content-heavy pages — they reduce perceived wait time and keep users engaged.



Loading State Checklist



Show loader immediately



Keep UI interactive where possible



Provide meaningful loader indicators



Handle success, error, and empty states



Hide loader quickly when done



Test on slow networks



Good loading states = Happy users + Better performance + Fewer support tickets

User Action • Loader • API Call • Data • UI



Error State

Handle failures gracefully and communicate issues clearly to users.



Why It Matters

- Things can go wrong — APIs fail, networks drop, or data is invalid.
- A good error state reduces frustration and builds trust.
- Clear messages help users take the right action.



Best Practices

- ✓ Show user-friendly, actionable messages
- ✓ Be specific, not generic
- ✓ Offer next steps or retry options
- ✓ Log technical errors (not shown to users)
- ✓ Keep the UI consistent and accessible

Error Handling Flow



Example: Angular Service (TypeScript)

```
getEmployees(): Observable<Employee[]> {  
  return this.http.get<Employee[]>(this.apiUrl)  
    .pipe(  
      catchError((error: HttpResponse) => {  
        let message = 'Something went wrong.';  
        if (error.status === 0) {  
          message = 'Network error. Please check  
            your connection.';  
        } else if (error.status === 500) {  
          message = 'Server error. Please try again  
            later.';  
        } else if (error.error?.message) {  
          message = error.error.message;  
        }  
        return throwError(() => new Error(message));  
      })  
    );  
}
```

Example: UI Error States

Network Error



Unable to connect to the server.
Please check your internet connection.



Server Error



Something went wrong on our end.
Please try again later.



No Data Found



No employees found.
Try adjusting your filters.



Common Error Types



Network / Connection Errors

No internet, timeout, DNS failure.



Server Errors (5xx)

Internal server issues.



Client Errors (4xx)

Bad request, unauthorized, forbidden, not found.



Validation Errors

Invalid input or business rule violations.



Pro Tip

- Use a global HTTP interceptor to handle and standardize errors.
- Don't expose technical details to end users.



Error State Checklist



Show meaningful messages



Provide retry or alternative actions



Handle all error scenarios



Keep the UI consistent and accessible



Log errors for monitoring



Test failure scenarios regularly



Good error states = Happy users + Fewer support tickets + Better user experience



Failure Happens



Handle Gracefully



Keep Users Informed



Build Trust



Empty State

Communicate that there's no data and guide users to take action.



Why It Matters

- Sets clear expectations
- Reduces confusion and frustration
- Encourages the next best action
- Improves overall user experience



Best Practices

- ✓ Use friendly and helpful copy
- ✓ Provide a clear next step
- ✓ Keep the design simple and clean
- ✓ Be consistent across the application
- ✓ Avoid showing empty state too early

Empty State Flow



Examples in an Angular App

Employees



No employees found

You haven't added any employees yet.

+ Add Employee

Tasks



No tasks yet

Get started by creating your first task.

+ Create Task

Notifications



No notifications

You're all caught up!
Check back later for updates.

Common Empty State Patterns



No Results

User searched or filtered but no results matched.



No Data Yet

No data has been created yet.



Nothing Here

Empty section with no content to display.



No Matches

Filters applied but no items match the criteria.



No Activity

No recent activity or updates available.



Pro Tip

- Combine empty states with helpful CTAs.
- Use illustrations to create a friendly feel.
- Ensure the empty state is accessible (colors, contrast, and text).



Great Empty States Are...



Friendly
Warm, human and approachable.



Helpful
Guide users on what to do next.



Action-Oriented
Encourage the right next action.



Clear
Easy to understand at a glance.



Consistent
Aligned with the overall design system.



Good empty states = Happy users + Better engagement + Clear next actions



Search



API



No Data



Empty State



UI

CHANGE DETECTION OVERVIEW

The mechanism that decides when to walk the component tree and actually update the DOM.



Default Strategy

Zone.js patches async browser APIs and triggers a broad check across the component tree.



OnPush Strategy

A component only re-checks when an @Input reference changes, an event fires inside it, or a signal it reads changes.



Signals Sharpen Both

Because signals report precisely what depends on them, they let Angular skip work even under the Default strategy.

KEY TAKEAWAY Change detection is the engine underneath every reactive concept covered today -- signals, the async pipe, and event bindings are all just different ways of telling this engine what to check.



Change Detection Overview

Angular automatically updates the view when your data changes.



What is Change Detection?

Change detection is Angular's mechanism to keep the UI (View) in sync with the application data (Model).

When data changes, Angular checks components and updates only what's needed in the DOM.



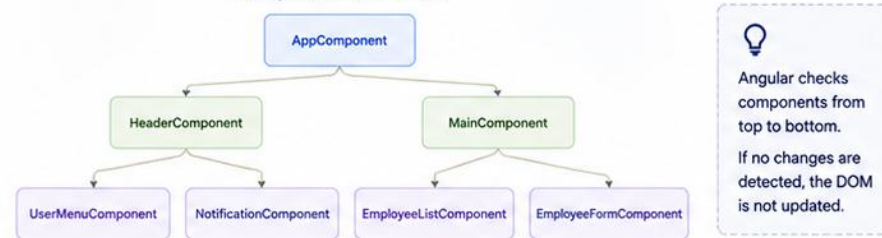
Key Characteristics

- ✓ Automatic and efficient
- ✓ Runs in a component tree
- ✓ Updates the DOM selectively
- ✓ Works with both Signals and Observables
- ✓ Optimized for performance

How Change Detection Works



Component Tree Check



Angular checks components from top to bottom. If no changes are detected, the DOM is not updated.

```
<app-root>
  <app-header></app-header>
  <app-main>
    <app-employee-list></app-employee-list>
    <app-employee-form></app-employee-form>
  </app-main>
</app-root>
```

Update Only What Changed

	Before Change	After Change
HeaderComponent	✓ No Change	✓ No Update
EmployeeListComponent	! Data Changed	✓ Updated
EmployeeFormComponent	✓ No Change	✓ No Update

What Triggers Change Detection?



User Events

Clicks, typing, navigation, form input, button clicks.



Async Operations

HTTP responses, promises, setTimeout, intervals.



Signals / Observables

When signals update or observables emit.



Input Property Changes

@Input() properties change in parent component.



Manual Triggers

detectChanges(), markForCheck(), ApplicationRef.tick().

Change Detection Strategies



Default (CheckAlways)

- Automatically checks every component in each change detection cycle.
- Easier but can be less performant.



OnPush (CheckOnce)

- Checks only when inputs change, an event occurs, or manually triggered.
- More performant for large applications.



Best Practices



Use OnPush

Use OnPush strategy for better performance.



Immutable Data

Prefer immutable updates to help Angular detect changes efficiently.



Track By in *ngFor

Use trackBy to prevent unnecessary DOM re-renders.



Avoid Manual DOM Manipulation

Let Angular manage the DOM.



Profile Performance

Use Angular DevTools to analyze change detection cycles.



Efficient change detection = Better performance + Smoother user experience



Model



Change Detection



Component Tree



View (DOM)

PERFORMANCE CONSIDERATIONS

Most Angular performance problems trace back to one root cause: too much unnecessary re-checking.



OnPush by Default

Set `changeDetection`:
`ChangeDetectionStrategy.OnPush` on new components -- especially presentational ones.



Signals Reduce Re-Checks

Fine-grained signal tracking means far fewer bindings get re-evaluated on any given change.



Measure Before Optimizing

Angular DevTools' profiler shows real re-render counts -- guess less, measure more.

KEY TAKEAWAY OnPush plus signals is the single highest-leverage performance decision available in modern Angular -- apply both by default, then measure before chasing anything more exotic.



Performance Considerations

Build fast, responsive, and scalable Angular applications.



Why Performance Matters

- Improves user experience
- Reduces load times and rendering delays
- Lowers CPU and memory usage
- Boosts SEO and accessibility
- Supports scalability and growth



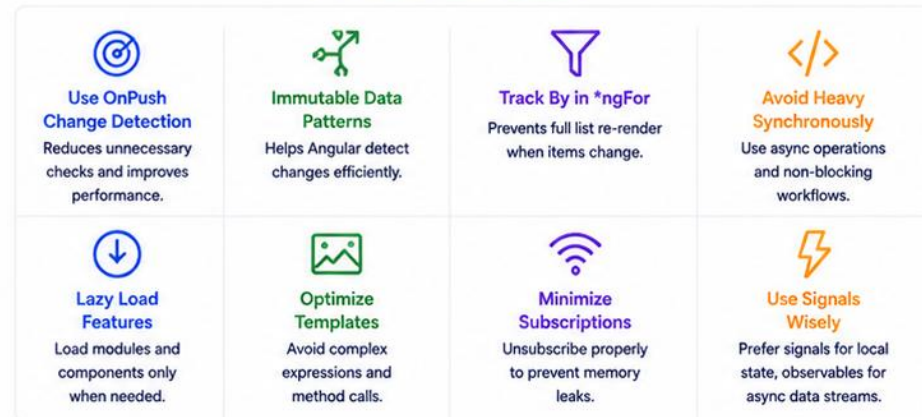
Key Areas to Optimize

- ✓ Change Detection
- ✓ Re-Renders
- ✓ DOM Updates
- ✓ Data Flow & State Management
- ✓ Bundle Size
- ✓ Network Requests

Change Detection & Rendering



Performance Best Practices



Common Performance Pitfalls



Using Default Change Detection Everywhere

Leads to unnecessary checks.



Mutating Objects/Arrays

Breaks change detection and causes bugs.



No Track By in *ngFor

Causes full DOM re-render on list changes.



Heavy Logic in Templates

Slows down rendering.



Leaking Subscriptions

Increases memory usage over time.

Measuring Performance



Angular DevTools

Analyze change detection and performance.



Lighthouse

Audit performance, accessibility, and SEO.



Browser DevTools

Profile CPU, memory, and network.



Performance Checklist



Use OnPush where possible



Use trackBy in all *ngFor loops



Avoid heavy logic in templates



Prefer async pipe and signals



Lazy load routes and modules



Profile and measure regularly



Remove unused code and dependencies.



Performance is not a feature — it's a foundation for a great user experience.



Faster Apps



Better UX



Lower Costs



Scalable Systems

AVOIDING UNNECESSARY RE-RENDERS

A short, concrete checklist -- apply these and most Angular performance problems never happen in the first place.



Always Use track in @for

@for (c of customers(); track c.id) -- without it, Angular may recreate DOM elements it did not need to.



Never Mutate @Inputs

Replace arrays/objects immutably (from Updating Signals) so OnPush's reference check actually works.



Prefer computed() Over Manual Sync

A computed signal only recalculates when its real dependencies change -- a manually-updated duplicate field can drift or over-fire.

KEY TAKEAWAY Three habits -- track, immutability, and computed() -- prevent nearly every avoidable re-render an Angular app is likely to hit.



Avoiding Unnecessary Re-Renders

Re-rendering updates the DOM when component state changes. Avoid unnecessary re-renders to improve performance, responsiveness, and user experience.

KEY TAKEAWAYS



Render Only What Changed

Update the UI only when data actually changes.



Use OnPush Change Detection

OnPush strategy reduces unnecessary change detection cycles.



Use Immutable Data Patterns

Immutable updates help Angular detect real changes efficiently.



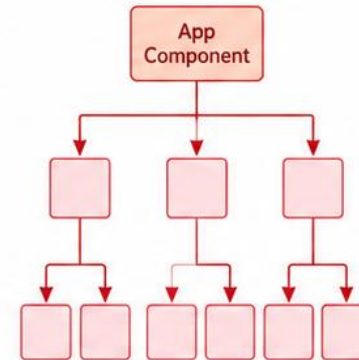
Optimize Templates

Avoid heavy computations, methods, and complex bindings in templates.



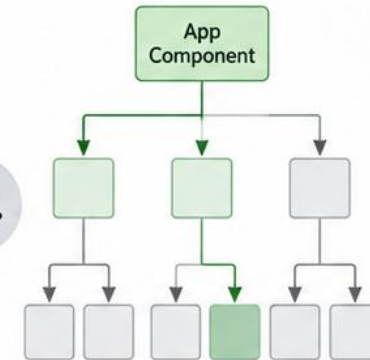
GOAL: Build fast, responsive Angular applications by rendering efficiently and updating only what's necessary.

UNNECESSARY RE-RENDERS



VS.

OPTIMIZED RE-RENDERS



Changes in one part trigger re-renders across the entire component tree.

Changes update only the necessary components, keeping the UI efficient.



WHY IT MATTERS

Fewer re-renders = better performance, lower CPU usage, and smoother user experience, especially in large applications.

ENTERPRISE STATE MANAGEMENT PATTERNS

The signals-and-services patterns from this module, applied consistently across a large, multi-team Angular app.



State Ownership by Domain

Each feature area owns and exposes its own signals through its own service -- no shared global blob.



Facade Services for State

A facade (from Module 33) can expose a simplified, read-only signal API over several underlying state services.



Immutability as a Team Rule

Large teams enforce the update-not-mutate discipline with lint rules, not just individual developer memory.

KEY TAKEAWAY Enterprise state management is this module's own patterns -- services, signals, immutability -- applied consistently and enforced automatically, not a fundamentally different toolkit.



Enterprise State Management Patterns

Choose the right pattern for the right problem.
Scalable Angular applications organize state by scope, ownership, and responsibility.

GUIDING PRINCIPLES



Single Source of Truth

Each piece of state should have one authoritative owner.



Scope by Feature

Encapsulate state within the feature that owns it.



Predictable Data Flow

Make state changes predictable and traceable.



Performance First

Minimize re-renders and unnecessary subscriptions.



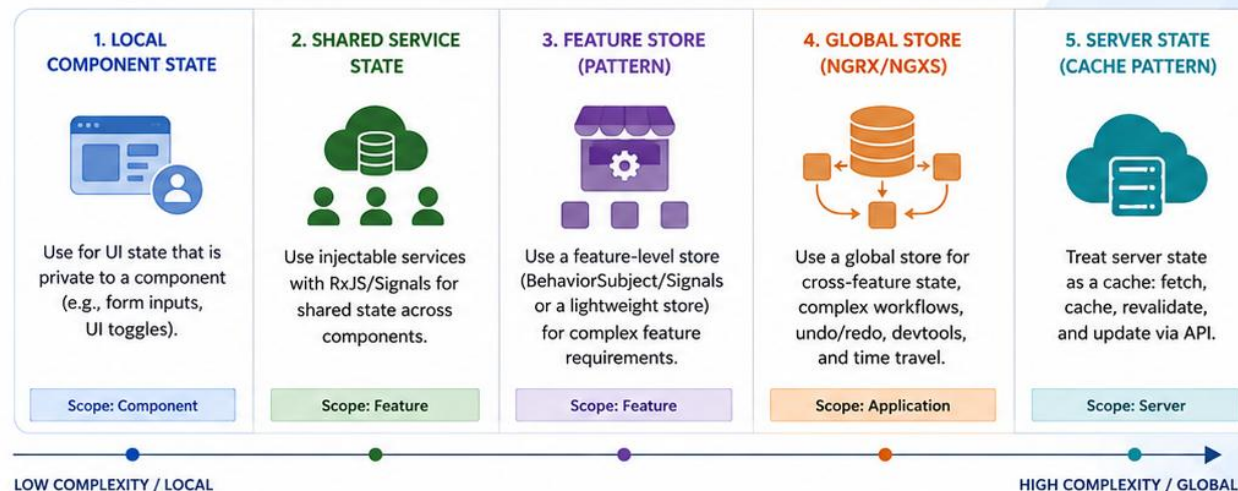
Testable and Maintainable

Patterns should simplify testing and long-term maintenance.



GOAL: Build scalable, maintainable, and high-performance Angular applications using the right state management patterns.

STATE MANAGEMENT PATTERNS IN ANGULAR



PATTERN COMPARISON

	PATTERN	BEST FOR	PROS	CONSIDERATIONS
1	Local Component State	Simple UI state	Simple, fast, easy to test	Not shareable
2	Shared Service State	Shared across a few components	Lightweight, easy to implement	Can grow complex over time
3	Feature Store	Complex feature state	Encapsulated, scalable within feature	Need discipline in design
4	Global Store (NgRx/NgXS)	Cross-feature, complex application state	Powerful, devtools, time travel, predictable	More boilerplate, learning curve
5	Server State (Cache)	Data from backend APIs	Keeps data fresh, consistent	Requires caching strategy

BEST PRACTICES



Choose the simplest pattern that solves the problem.



Keep state close to where it is used.



Avoid dumping everything in a global store.



Monitor performance and re-render behavior.



Document state ownership and data flow.

SHARED STATE WITH SERVICES

The pattern this whole module has been building toward: a service, private writable signals, public readonly signals.



Private Writable Signal

private readonly _customers =
signal<Customer[]>([]); -- only the service
itself can call .set()/.update().



Public Readonly Signal

readonly customers =
this._customers.asReadonly(); -- any
component can read, none can write directly.



Methods Own the Mutations

loadCustomers(), addCustomer() -- every
state change happens through an explicit,
named service method.

KEY TAKEAWAY This single pattern -- private writable, public readonly, mutations only through named methods -- is the load-bearing shape of shared state across the entire Angular ecosystem.



Shared State with Services

Angular services are ideal for managing and sharing state across multiple components. Services provide a single source of truth and help keep state consistent, predictable, and testable.

WHEN TO USE



Shared across components

State needs to be accessed in multiple components.



Survives routing

State persists while navigating between routes.



Centralized business state

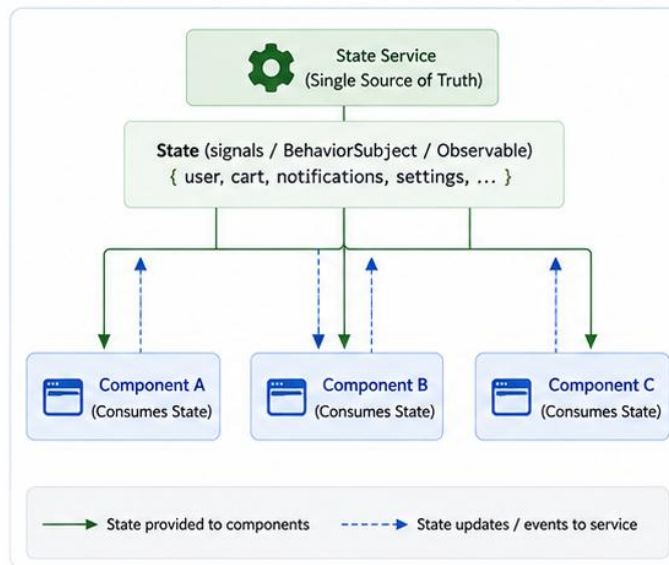
Manage application data like user, cart, notifications, settings, etc.



Easy to test and mock

Service state can be mocked in unit tests for components.

HOW IT WORKS



KEY BENEFITS

- ✓ **Single Source of Truth** – Consistent state across the app
- ✓ Simplifies communication – No need for @Input/@Output across many levels
- ✓ Maintains state during navigation
- ✓ Easier testing and mocking
- ✓ Scales well for medium-complexity applications

BEST PRACTICES

- ⚠ Keep services focused and cohesive
- ⚠ Avoid putting component-only state in services
- ⚠ Expose state as Observables/Signals, not mutable variables
- ⚠ Use immutable updates for complex objects
- ⚠ Clear state when no longer needed

EXAMPLE: USER SERVICE WITH BEHAVIOBSUBJECT

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private userSubject = new BehaviorSubject<User | null>(null);
  public user$ = this.userSubject.asObservable();

  // Get current user value
  get user(): User | null {
    return this.userSubject.value;
  }

  // Update user
  setUser(user: User | null): void {
    this.userSubject.next(user);
  }

  // Clear user (logout)
  clearUser(): void {
    this.userSubject.next(null);
  }
}
```

USAGE IN COMPONENTS

```
export class HeaderComponent {
  user$ = this.userService.user$;
  constructor(private userService: UserService) {}

  logout() {
    this.userService.clearUser();
  }
}
```

COMMON USE CASES



User Session



Shopping Cart



Notifications



App Settings



Global Filters



GOAL: Use Angular services to manage and share application state efficiently, ensuring consistency and maintainability.

INTRODUCTION TO LARGER STATE SOLUTIONS

For most apps, services plus signals are enough -- but some applications outgrow that and reach for more structure.



NgRx (Redux Pattern)

A single global store, actions, and reducers -- powerful, but real added ceremony for smaller apps.



@ngrx/signals (SignalStore)

A newer, signals-native alternative to classic NgRx -- less boilerplate, same disciplined structure.



Know When You Need It

If services-plus-signals is handling crm-ui's state cleanly, adding NgRx is very likely solving a problem you do not have yet.

KEY TAKEAWAY Reach for NgRx or a SignalStore when many independent teams need one predictable, centrally-inspectable state shape -- not as a default starting point for every Angular app.

TRY IT YOURSELF — EXERCISE 6: PLAN UI STATES AND SELF-CHECK

Pre-lab gate: define the loading, empty, and error states Lab 34 grades, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab34-readiness.md)

Step	What To Do
1 — Three States	Describe exactly what renders while loading, when empty, and on error.
2 — Tell Them Apart	Say how a user distinguishes empty from error without reading a console.
3 — Re-render Check	Name one change that would cause avoidable re-renders and how you avoid it.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Three distinct states

```
loading : spinner, no list, no message
empty   : 'No customers match this filter.' + clear-filter action
error    : banner with a retry -- never a silent blank screen
```

TRY IT NOW — Complete Exercise 6 before Lab 34: exercises/exercise-06-lab34-readiness.md (workspace: examples/module-34-exercises).



Introduction to Larger State Solutions

For complex applications, dedicated state management libraries provide scalability, better tooling, and predictable state. Use them when shared state grows beyond simple service patterns.

WHY CONSIDER LARGER SOLUTIONS?



Complexity

Many features, deep component trees, and cross-cutting state.



Predictability

Centralized state and unidirectional data flow reduce bugs.



Tooling & Dev Experience

Time-travel debugging, state inspection, and better tooling.



Maintainability & Scalability

Easier to evolve as the application grows.

WHEN TO USE WHAT?



SMALL APPS

Use Angular Services or Component State (Signals)

Simple, local, or isolated state.



MEDIUM APPS

Use Services with RxJS/Signals

Shared state across features, moderate complexity.



LARGE APPS

Use NGXS or NgRx

Complex workflows, cross-feature state, need tooling.



ENTERPRISE APPS

Use NgRx

Maximum scalability, predictability, advanced DevTools.

POPULAR STATE MANAGEMENT SOLUTIONS

FEATURE / CRITERIA	 NGX (NgRx)	 NGXS	 NG-ZORRO ZUSTAND (with Signals)	 ANGULAR SIGNALS (Built-in)
State Management Style	Redux-inspired, Immutable, Unidirectional	Action-based, Mutable or Immutable	Store with Signals, Lightweight	Fine-grained Reactive State
Learning Curve	High	Medium	Medium-Low	Low
Boilerplate	High	Medium	Low	Very Low
DevTools Support	Excellent	Good	Basic (Planned)	Basic
Time-travel Debugging	Excellent	Good	Limited	Limited
Best For	Large enterprise apps with complex workflows	Medium to large applications	Modern Angular apps wanting simplicity	Small to medium apps, component-level state
Ecosystem & Community	Very Large	Large	Growing	Built-in (Angular Core)
Immutability	Yes (Recommended)	Optional	No (Mutable with Signals)	N/A (Signal-based)

KEY CONSIDERATIONS

- ✓ Start simple. Add complexity only when needed.
- ✓ Consistency in patterns is more important than the library.
- ✓ Plan state boundaries around business domains, not components.
- ✓ Keep global state minimal – derive and compute when possible.
- ✓ Combine solutions: Signals for local state, NgRx/NGXS for global.



GOAL: Choose the right state management approach to build scalable, maintainable, and high-performing Angular applications.

COMMON STATE MANAGEMENT MISTAKES

The same handful of mistakes account for most avoidable state and reactivity bugs in real Angular code.

Mistake	Fix
Mutating an array/object signal in place	Always produce a new array/object in .update() -- see Updating Signals.
Forgetting {{ value() }} parentheses in a template	Signals must be called as functions everywhere they are read, templates included.
Using effect() to derive a value	Use computed() instead -- effect() is for side effects only, never for producing state.
Never handling the error callback	Every .subscribe() on a real HTTP call needs an error path -- silent failure looks like a stale UI.
Reaching for NgRx before it is needed	Start with a signal-based service; adopt a larger store only when a specific pain justifies it.

KEY TAKEAWAY Every mistake in this table has already been covered somewhere earlier in the module -- treat this table as the module's own final review, not new material.



Common State Management Mistakes

Avoid these pitfalls to build scalable, maintainable, and high-performance Angular applications.

REMEMBER

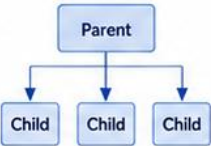
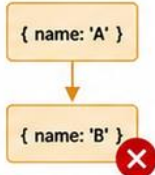


Good state management leads to predictable data flow, fewer bugs, better performance, and happier users.

STATE MANAGEMENT CHECKLIST

- ✓ Keep state close to where it's used
- ✓ Use the right tool for the right job
- ✓ Avoid duplication and tight coupling
- ✓ Keep state immutable and predictable
- ✓ Clean up subscriptions and effects
- ✓ Plan for scalability early
- ✓ Test state logic in isolation

TOP MISTAKES AND HOW TO AVOID THEM

1 OVERUSING @INPUT AND @OUTPUT  Problem Creates deep component trees and tight coupling, hard to maintain. <pre><!-- Passing through many layers --> <child-a [data]="data" (update)="onUpdate(\$event)"> <child-b [data]="data" (update)="onUpdate(\$event)"> <child-c> </child-b> </child-a></pre> ✓ Better Use a shared service or state library to avoid prop drilling. 	2 MUTATING STATE DIRECTLY  Problem Breaks change detection, leads to unpredictable UI updates. <pre>this.user.name = 'B'; // UI may not update</pre> ✓ Better Use immutable updates. <pre>this.user = { ...this.user, name: 'B' };</pre> 	3 SUBSCRIBING WITHOUT CLEANUP  Problem Leads to memory leaks and unexpected behavior in long-lived apps. <pre>this.service.getData() .subscribe(data => { this.data = data; });</pre> ✓ Better Unsubscribe properly or use async pipe / takeUntil. <pre>this.service.getData() .pipe(takeUntil(this.destroy\$)) .subscribe(data => this.data = data);</pre> 	4 KEEPING TOO MUCH STATE GLOBALLY  Problem Global state becomes a dumping ground, hard to understand and test. <pre>Global Store ├── X ├── Y └── Z</pre> ✓ Better Keep global state minimal and domain-focused. 	5 DERIVING STATE IMPROPERLY  Problem Stores duplicate data, causing inconsistency and sync issues. <pre>this.total = 0; this.items.forEach(i => this.total += i.price); // Must remember to update total</pre> ✓ Better Derive state when needed (computed signals / selectors). <pre>readonly total = computed(() => this.items().reduce((sum, i) => sum + i.price, 0));</pre> 	6 IGNORING PERFORMANCE  Problem Unnecessary re-renders and heavy computations slow down the app. <pre>{{ getFormattedDate(date) }} <!-- Function called on every change detection --></pre> ✓ Better Use pure pipes, computed signals, and OnPush change detection. 
---	---	--	---	--	---



KEY TAKEAWAY

Choose the right pattern, keep state predictable, and prioritize maintainability and performance. Simple today, scalable tomorrow!



USE THE RIGHT TOOL

- Simple & Local: Component State / Signals
- Shared & Cross-Component: Services / RxJS / Signals
- Complex & Global: NgRx / NGXS
- Server Data: Server State (cache patterns)

LAB OVERVIEW — STATE AND EVENT MANAGEMENT IN ANGULAR

Drive CRM list/filter UI with Signals, computed values, and reactive form state /

BUSINESS SCENARIO

- List/filter state lives in Signals. Do not leave a forever spinner or blank screen when a filter matches nothing. Seeds stay Amina (`CUS-1001`) and Ravi (`CUS-1002`).

LEARNING OBJECTIVES

- Create, read, and update Signals; derive with `computed`; observe with `effect` Explain Signals for UI state vs Observables for async streams Build a basic reactive filter form Render loading, empty, and error states consistently

WHAT YOU BUILD

- `examples/lab34-crm-ui/` — Signals store/page, filter form, state notes | Filterable list (CUS-1001 / CUS-1002) with loading · empty · error branches |

KEY TAKEAWAY Make the CRM list ****reactive****: hold customers in Signals, derive filters with `computed`, contrast with RxJS Observables, drive criteria with a reactive form, and show ****loading / empty / error**** so Lab 35 can plug HttpClient into the same branches.



Lab Overview – State and Event Management

Build an Angular feature to practice component state, signals, observables, forms, and event communication for a real-world user experience.



What You Will Build

An Employee Management feature with shared state, reactive UI, forms and event-driven updates.



Key Concepts Practiced

- ✓ Component local state
- ✓ Shared state with services
- ✓ Signals and computed signals
- ✓ Event binding and custom events
- ✓ Reactive forms and validation
- ✓ Loading and error states
- ✓ RxJS observables and BehaviorSubject
- ✓ Parent-child communication



Tools & Environment

- Angular 17+ (Standalone Components)
- TypeScript
- Angular CLI



Deliverables

- Working Angular feature module
- Employees list with add/edit functionality
- Shared state service using Signals / RxJS
- Reactive form with validation
- Clean, reusable component design



Goal

Implement predictable, scalable and responsive UI behavior using modern Angular state and event management techniques.

Employee Portal

[Dashboard](#)
[Employees](#)
[Departments](#)
[Notifications](#) 3
[Settings](#)

Employee Management

[+ Add Employee](#)

Total Employees
 124

Active
 112

On Leave
 6

New This Month
 6

All Departments ▾

Name	Department	Status	Actions
Amit Sharma	Engineering	Active	
Priya Singh	Product	Active	
Rahul Kumar	Operations	On Leave	
Anita Desai	Finance	Active	



Success Criteria

- UI updates automatically on state changes
- Events flow correctly between components
- Forms validate and show proper error messages
- Code follows Angular best practices

LAB TASKS AND DELIVERABLES

State and Event Management in Angular — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy/scaffold lab34-crm-ui.
3	Implement Signals + filter form + three UI states.
4	Smoke ng serve; evidence under notes/screenshots/lab-34/.
5	Mark timed-path Pass; finish remaining Steps as homework.
6	lab34-crm-ui under examples/
7	Signal-based customers including CUS-1001 / CUS-1002
8	computed filtered list driven by reactive form (or filter Signal)
9	Loading, empty, and error UI demonstrated
10	docs/state-notes.md (Signals vs Observables)
11	Screenshots of filter + empty or error
12	Serve/build evidence

EXPECTED DELIVERABLES

- lab34-crm-ui under examples/ Signal-based customers including CUS-1001 / CUS-1002 computed filtered list driven by reactive form (or filter Signal) Loading, empty, and error UI demonstrated docs/state-notes.md (Signals vs Observables) Screenshots of filter + empty or error Serve/build evidence No node_modules/ / secrets committed

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 34; complete pre-lab exercises 1→6 first, then the graded lab.

Lab Tasks and Deliverables

In this lab, you will build an Angular application that demonstrates state management, event handling, and reactive UI patterns using Signals, RxJS, and Services.

LAB APPLICATION OVERVIEW



LAB TASKS

- 1 Project Setup**
Set up the Angular project and organize feature-based folders. 
- 2 Shared State Service**
Create UserService using Signals to manage users and expose computed signals. 
- 3 User List Component**
Build a UserListComponent to display users with search and filtering using computed signals. 
- 4 Add User Component**
Create an AddUserComponent with a reactive form and validation. Add user and update shared state. 
- 5 Edit / Delete User**
Enable edit and delete actions with confirmation. State updates should reflect in all components. 
- 6 Parent-Child Communication**
Use @Input() and @Output() to pass data and events between components. 
- 7 Notifications with RxJS**
Implement a NotificationService using RxJS BehaviorSubject. Show toast/alert messages on actions. 
- 8 Loading, Error, Empty States**
Display appropriate UI states for loading, error, and no data. 
- 9 Optimize Performance**
Use OnPush change detection, computed signals, and trackBy to avoid unnecessary re-renders. 
- 10 Test State Flow**
Verify state updates, event communication, and UI reactivity. Ensure data consistency across components. 

DELIVERABLES

- ✓ Working Angular application with all features
- ✓ UserService using Signals with computed state
- ✓ UserListComponent with search & filter
- ✓ Add/Edit form with validation
- ✓ Event communication between components
- ✓ Global notifications using RxJS BehaviorSubject
- ✓ Loading, error, and empty state handling
- ✓ Optimized UI with minimal re-renders
- ✓ Clean, modular, and reusable code
- ✓ GitHub repository with README



EVALUATION CRITERIA

- | | |
|---|-----|
| ★ Correct state management with Signals | 25% |
| ★ Event handling and communication | 20% |
| ★ UI reactivity and user experience | 20% |
| ★ Code quality and component design | 20% |
| ★ Best practices and performance | 15% |

TIPS FOR SUCCESS

- Keep state minimal and source of truth in services.
- Prefer Signals for local/shared state, RxJS for async events.
- Use computed signals for derived state.
- Avoid unnecessary subscriptions and side effects.
- Test incrementally as you build each feature.



SUCCESS: You will build a reactive Angular application that demonstrates clean state management, effective event handling, and a great user experience.

 Real-World Skills

 Best Practices

 Job-Ready

MODULE 34 SUMMARY

Signals for synchronous state, RxJS for async streams, and events tie the whole loop together.

KEY CONCEPTS COVERED



Signals

signal(), computed(), and effect() -- Angular's synchronous, fine-grained reactive primitive.



RxJS Observables

Streams over time, Subjects, and BehaviorSubject -- essential for HTTP and truly async data.



Choosing the Right Tool

Signals for 'what is the value now'; RxJS for 'when, in what order, and can it be cancelled.'



Event-Driven State Updates

Every UI interaction flows: event -> handler -> signal update -> automatic re-render.



Reactive Forms

FormControl/FormGroup model value, validity, and touch state explicitly and testably.



Performance Habits

OnPush, track, and immutable updates prevent nearly every avoidable re-render.

MODULE FLOW RECAP

1

1. Signals

2

2. RxJS

3

3. Events & Forms

4

4. Async States

5

5. Performance

KEY TAKEAWAY Model state as signals by default, reach for RxJS when timing genuinely matters, and always update immutably -- that combination is most of what makes an Angular UI feel reliably reactive.



Module Summary

In this module, you learned how to manage application state and handle events in Angular using modern reactive patterns. You explored Signals, RxJS, and Services to build maintainable, scalable, and highly responsive applications.



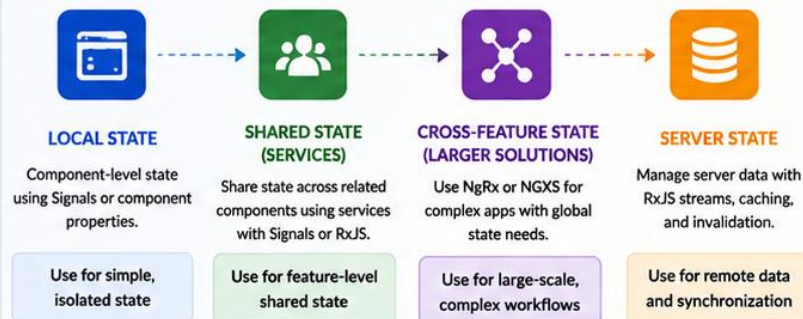
KEY OUTCOMES

- ✓ Understand local vs shared state and when to use each.
- ✓ Use Signals for reactive, fine-grained component state.
- ✓ Use RxJS for asynchronous data streams and complex event flows.
- ✓ Manage UI and form state effectively with reactive patterns.
- ✓ Apply enterprise patterns to build predictable and maintainable UIs.

WHAT WE COVERED



ENTERPRISE STATE MANAGEMENT PATTERNS



KEY TAKEAWAYS

- ✓ Choose the right tool: Signals, RxJS, or both.
- ✓ Keep state minimal, predictable, and source of truth clear.
- ✓ Use services for sharing state; avoid tight coupling between components.
- ✓ Prefer reactive patterns for asynchronous and event-driven UI.
- ✓ Optimize performance by reducing unnecessary re-renders.
- ✓ Follow best practices for scalable and maintainable state management.



MODULE ACHIEVEMENT: You can now manage state and events effectively in Angular applications using modern reactive patterns and enterprise-ready approaches.



Practical Skills



Real-World Patterns



Job-Ready

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of signals and RxJS fundamentals.

1. What does `count()` return if `count = signal(5)`?

- A. The signal object itself
- B. 5**
- C. undefined
- D. A Promise resolving to 5

3. What should `effect()` be used for?

- A. Deriving a new value from other signals
- B. Side effects, like logging or persisting data**
- C. Replacing `computed()` in most cases
- D. Reading a signal's value in a template

2. Why must `customers.update()` always produce a brand-new array?

- A. Arrays cannot be stored in signals otherwise
- B. Change detection checks the array's reference**
- C. TypeScript requires it for type safety
- D. It only matters for objects, not arrays

4. Why is an Observable from `http.get()` described as 'lazy'?

- A. It runs slower than a Promise
- B. No request is sent until something subscribes**
- C. It only works after `ngOnInit`
- D. It requires the async pipe to function at all

KEY TAKEAWAY Signals are read by calling them; array/object updates must replace the reference; `effect()` is for side effects only; Observables do nothing until subscribed.

KNOWLEDGE CHECK (2 OF 2)

Test your understanding of events, forms, async state, and performance.

5. In the parent-child event flow, what typically happens right after a parent's event handler runs?

- A. The child component is destroyed
- B. A signal updates; dependent bindings re-render**
- C. The page reloads
- D. Nothing, unless detectChanges() is called manually

7. After a successful load, a customer list is empty. What is the correct UI response?

- A. Show the same error message used for failed requests
- B. Show nothing at all
- C. Show a distinct empty-state message**
- D. Keep showing the loading spinner indefinitely

6. Which Angular form approach is best suited for a form with several fields and real validation rules?

- A. Template-Driven Forms
- B. Reactive Forms**
- C. Plain HTML forms only
- D. RxJS Subjects alone

8. Which habit most directly helps OnPush change detection actually skip unnecessary work?

- A. Mutating arrays in place for speed
- B. Avoiding the track expression in @for
- C. Always using the Default change detection strategy
- D. Replace arrays/objects immutably on update**

KEY TAKEAWAY Event handlers update signals, which re-render automatically; Reactive Forms fit complex validation; empty state needs its own message; immutable updates are what make OnPush actually work.



Knowledge Check



Let's verify your understanding of the key concepts covered in this module.

1

What is REST?

Choose the best description of REST.

2

Which HTTP method should be used to create a new resource?

Choose the correct HTTP method.

3

What does a 404 status code indicate?

Choose the best answer.

4

Which REST constraint ensures that each request contains all the information needed?

Choose the correct constraint.

5

Which status code represents a successful resource creation?

Choose the correct status code.

6

What is the purpose of API versioning?

Choose the best answer.



How to use this slide

Review each question and select the answer that best fits your understanding.



Why it matters

Strong fundamentals in REST API design ensure scalable, reliable, and maintainable enterprise applications.



*"State is not what your app has -- it is what
your app remembers between one render
and the next."*

MODULE 34 COMPLETE

State and Event Management in Angular

You can now manage state with signals and RxJS, and wire events cleanly across a component tree.

